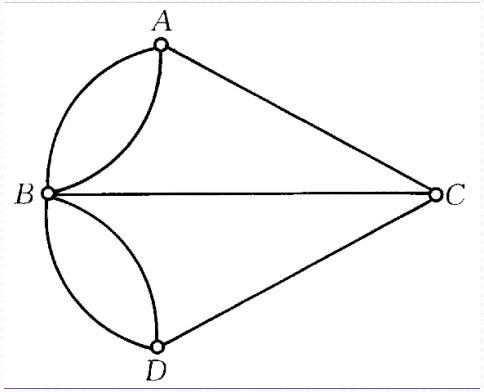
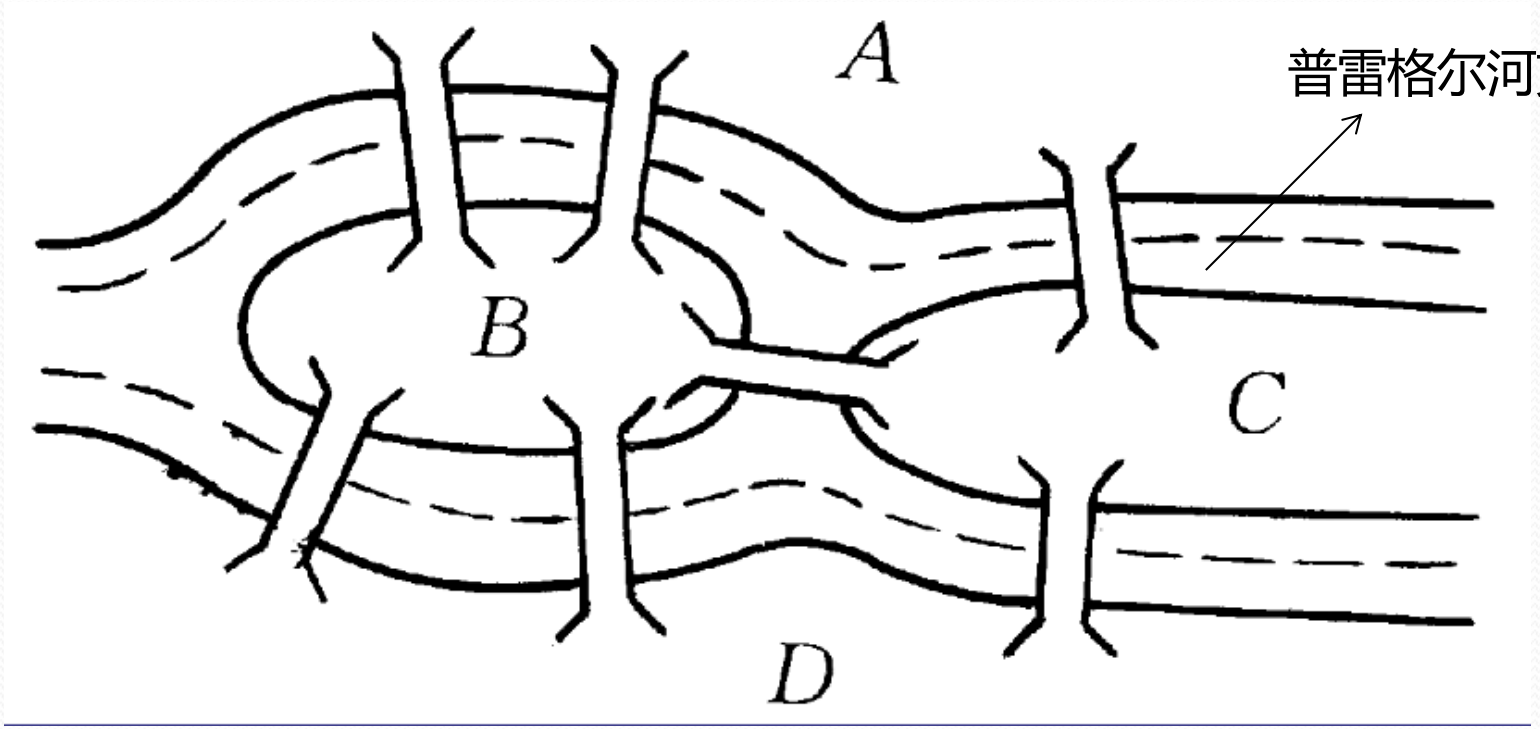
Chapter 6 Graphs
第六章　图

图论研究的先河

- 著名的哥尼斯堡七桥问题





Chapter Summary

- Graphs and Graph Models 图和图模型
- Graph Terminology and Special Types of Graphs
- Representing Graphs and Graph Isomorphism
- Connectivity
- Euler and Hamiltonian Graphs
- Shortest-Path Problems (*not currently included in overheads*)
- Planar Graphs (*not currently included in overheads*)
- Graph Coloring (*not currently included in overheads*)

Graphs and Graph Models
图和图模型

Section Summary

- Introduction to Graphs
- Graph Taxonomy
- Graph Models

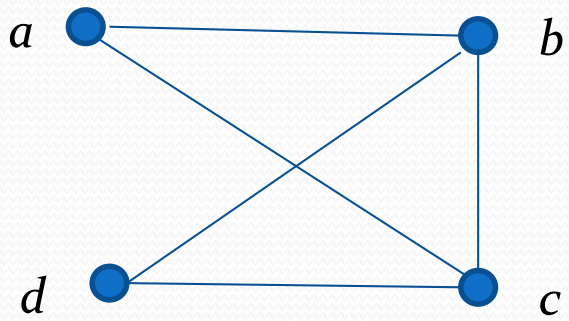
Graphs

Definition: A *graph* $G = (V, E)$ consists of a nonempty set V of *vertices* (or *nodes*) and a set E of *edges*. Each edge has either one or two vertices associated with it, called its *endpoints*. An edge is said to *connect* its endpoints.

定义：一个图G由顶点的非空集V和边集E构成，每条边有一个或两个顶点与它相连，这样的顶点成为边的端点。边连接它的端点。

Example:

This is a graph with four vertices and five edges.



Remarks:

- A graph with an infinite vertex set is called an *infinite graph*. A graph with a finite vertex set is called a *finite graph*. We (following the text) restrict our attention to finite graphs.
- 图G的顶点集V可能是无限的。顶点集为无限集的图称为无限图，与之相对，顶点集为有限集的图称为有限图。在本书中，通常只考虑有限图。

Some Terminology

- In a *simple graph* each edge connects two different vertices and no two edges connect the same pair of vertices.
- 在简单图中，每条边都连接两个不同的顶点且没有不同的两条边连接着相同的一对顶点。
- *Multigraphs* may have multiple edges connecting the same two vertices. When *m* different edges connect the vertices *u* and *v*, we say that {*u*,*v*} is an edge of *multiplicity m*.
- 多重图可能会有多重边连接同一对顶点的图。当有***m***条不同的边与相同的无序顶点对{***u***, ***v***}相关联，{***u***, ***v***}是一条多重度为***m***的边。

Some Terminology

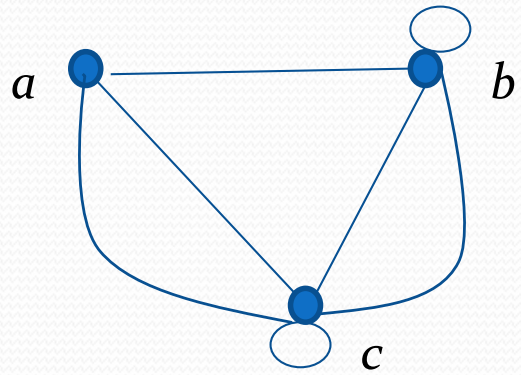
- An edge that connects a vertex to itself is called a *loop*.

  把一个顶点连接到它自身的边的图，这样的边称为环。

- A *pseudograph* may include loops, as well as multiple edges connecting the same pair of vertices.

  伪图可能包含环、还有可能包含连接同一对顶点的多重边的图。

Example:
This pseudograph has both multiple edges and a loop.



Remark: There is no standard terminology for graph theory. So, it is crucial that you understand the terminology being used whenever you read material about graphs.

Directed Graphs

Definition: An *directed graph* (or *digraph*) $G = (V, E)$ consists of a nonempty set V of *vertices* (or *nodes*) and a set E of *directed edges* (or *arcs*). Each edge is associated with an ordered pair of vertices. The directed edge associated with the ordered pair (u,v) is said to *start at* u and *end at* v.

定义**2**：一个有向图(V,E)由一个非空顶点集V和一个有向边（或弧）集E组成。每条有向边与一个顶点有序对相关联。认为与有序对(u,v)相关联的有向边开始于u、结束于v。

Remark:

- Graphs where the end points of an edge are not ordered are said to be *undirected graphs*.

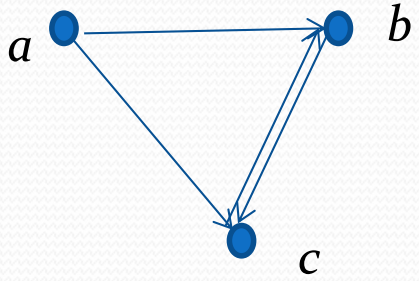
Some Terminology (*continued*)

- A *simple directed graph* has no loops and no multiple edges.

当一个有向图不包含环和多重有向边时，就称为**简单有向图**。

Example:

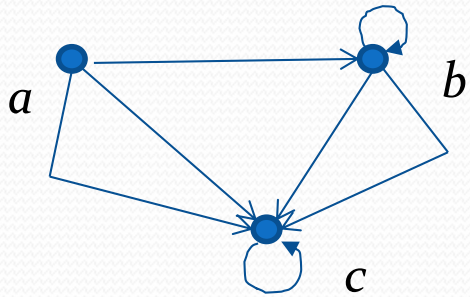
This is a directed graph with three vertices and four edges.



- A *directed multigraph* may have multiple directed edges. When there are *m* directed edges from the vertex *u* to the vertex *v*, we say that (*u*,*v*) is an edge of *multiplicity m*.
- **有向多重图**是包含多重有向边的有向图。当m条有向边中的每一条都与一对有序顶点对(*u*,*v*)相关联，则(*u*,*v*)是一条多重度为*m*的边。

Example:

In this directed multigraph the multiplicity of (*a*,*b*) is 1 and the multiplicity of (*b*,*c*) is 2.



Graph Terminology: Summary

- To understand the structure of a graph and to build a graph model, we ask these questions:

  Are the edges of the graph undirected or directed (or both)?

  - If the edges are undirected, are multiple edges present that connect the same pair of vertices? If the edges are directed, are multiple directed edges present?
  - Are loops present?

以下三个问题能够帮助我们理解图的结构和构建图模型：

- **1.** 图的边是有向的还是无向的（还是两者皆有）？
- **2.**如果是无向图，是否存在连接相同顶点对的多重边？如果是有向图，是否存在多重有向边？
- **3.** 是否存在环？

TABLE 1 **Graph Terminology.**

Type	*Edges*	*Multiple Edges Allowed?*	*Loops Allowed?*
Simple graph	Undirected	No	No
Multigraph	Undirected	Yes	No
Pseudograph	Undirected	Yes	Yes
Simple directed graph	Directed	No	No
Directed multigraph	Directed	Yes	Yes
Mixed graph	Directed and undirected	Yes	Yes

Graph Models 图模型

Social networks 社交网络

朋友关系图：无向边、无多重边也无环

影响图：用顶点表示每个人，用有向边表示

学术合作图：为学术论文的合作者关系建模。顶点表示人，用边连接两个顶点表示的两个合写论文的人。

Communications networks 通信网络

网络图

引文网络

Information networks 信息网络

Software design 软件设计

模块依赖图

优先图

Transportation networks 交通网络

机场网

路线网

Biological networks 生物网络

蛋白质相互作用图

Graph Terminology and Special Types of Graphs
图的术语和几种特殊的图

Section Summary

- Basic Terminology
- Some Special Types of Graphs
- Bipartite Graphs
- Bipartite Graphs and Matchings (*not currently included in overheads*)
- Some Applications of Special Types of Graphs (*not currently included in overheads*)
- New Graphs from Old

Basic Terminology

Definition 1. Two vertices *u*, *v* in an undirected graph *G* are called *adjacent* (or *neighbors*) in *G* if there is an edge *e* between *u* and *v*. Such an edge *e* is called *incident with* the vertices *u* and *v* and *e* is said to *connect u* and *v*.

定义1 若***u***和***v***是无向图***G***中的一条边的端点，则称两个顶点***u***和***v***称为在***G***里邻接（或相邻）。若***e***和***{u，v}***相联系，则边***e***称为关联顶点***u***和***v***。也可以说边***e***连接***u***和***v***。

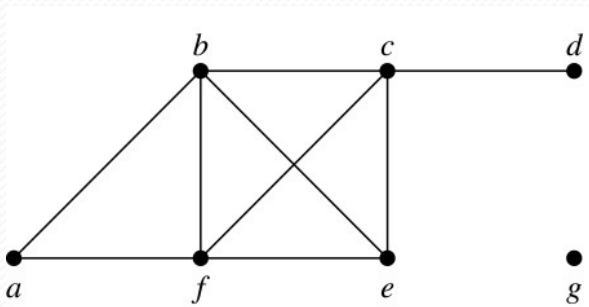
Definition 2. The set of all neighbors of a vertex *v* of $G = (V, E)$, denoted by $N(v)$, is called the *neighborhood* of *v*. If *A* is a subset of *V*, we denote by $N(A)$ the set of all vertices in *G* that are adjacent to at least one vertex in *A*. So,

定义2 ***G***中的一个顶点***v***的所有邻接顶点的集合记为***N(v)***。如果***A***是***V***的一个子集，***N(A)*** 则表示与***A***中至少一个顶点相邻的所有顶点的集合。

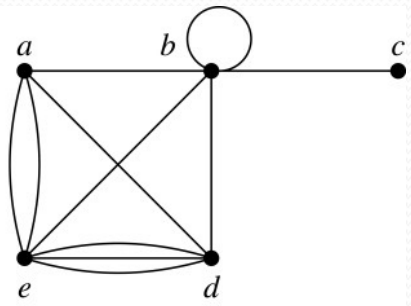
$$N(A) = \bigcup_{v \in A} N(v).$$

- **Definition 3**. The *degree of a vertex in a undirected graph* is the number of edges incident with it, except that a loop at a vertex contributes two to the degree of that vertex. The degree of the vertex v is denoted by $\deg(v)$.
- **定义3 在无向图里顶点的度是与该顶点关联的边的数目，例外的情形是，顶点上的环为顶点的度做出双倍贡献。顶点v的度表示成$\deg(v)$。**

Example: What are the degrees and neighborhoods of the vertices in the graphs G and H?



G



H

Degrees and Neighborhoods of Vertices

Solution:

G: $\deg(a) = 2, \deg(b) = \deg(c) = \deg(f) = 4, \deg(d) = 1,$
$\deg(e) = 3, \deg(g) = 0.$
$N(a) = \{b, f\}, N(b) = \{a, c, e, f\}, N(c) = \{b, d, e, f\}, N(d) = \{c\},$
$N(e) = \{b, c, f\}, N(f) = \{a, b, c, e\}, N(g) = \varnothing.$

把度为0的顶点称为孤立的。孤立点不与任何顶点相邻。

顶点是悬挂的，当且仅当它的度是1。

H: $\deg(a) = 4, \deg(b) = \deg(e) = 6, \deg(c) = 1, \deg(d) = 5.$
$N(a) = \{b, d, e\}, N(b) = \{a, b, c, d, e\}, N(c) = \{b\},$
$N(d) = \{a, b, e\}, N(e) = \{a, b, d\}.$

- 所有顶点的度求和是什么？跟边数的关系？



定理1 握手定理 设$G=(V,E)$是有e条边的无向图，则

$$2e = \sum_{v \in V} \deg(v)$$

注意，即使出现多重边和环，这个式子也仍然成立。

证明：每条边都为顶点的度之和贡献了2，因为一条边恰好关联2个（可能相同）顶点。

可以将一条边上有两个顶点类比为一次握手涉及到两只手。

Degrees of Vertices

Theorem 1 (*Handshaking Theorem*): If $G = (V,E)$ is an undirected graph with m edges, then

$$2m = \sum_{v \in V} \deg(v)$$

Proof:

Each edge contributes twice to the degree count of all vertices. Hence, both the left-hand and right-hand sides of this equation equal twice the number of edges. ◀

Think about the graph where vertices represent the people at a party and an edge connects two people who have shaken hands.

Handshaking Theorem

Example: How many edges are there in a graph with 10 vertices of degree six?

一个具有**10**个顶点而且每个顶点的度都为**6**的图，有多少条边？

Solution: Because the sum of the degrees of the vertices is $6 \cdot 10 = 60$, the handshaking theorem tells us that $2m = 60$. So the number of edges $m = 30$.

Example: If a graph has 5 vertices, can each vertex have degree 3?

如果一个图有5个顶点，那么可能存在每个顶点的度都为3的情况吗？

Solution: This is not possible by the handshaking thorem, because the sum of the degrees of the vertices $3 \cdot 5 = 15$ is odd.

- 握手定理说明无向图顶点的度之和是偶数。

Theorem 2: An undirected graph has an even number of vertices of odd degree.

定理2 无向图有偶数个奇数度顶点。

***Proof*:** Let V_1 be the vertices of even degree and V_2 be the vertices of odd degree in an undirected graph $G = (V, E)$ with m edges. Then

even $\longrightarrow$ $$2m = \sum_{v \in V} \deg(v) = \sum_{v \in V_1} \deg(v) + \sum_{v \in V_2} \deg(v).$$

must be even since $\deg(v)$ is even for each $v \in V_1$

This sum must be even because $2m$ is even and the sum of the degrees of the vertices of even degrees is also even. Because this is the sum of the degrees of all vertices of odd degree in the graph, there must be an even number of such vertices.

Directed Graphs

Definition: An *directed graph* $G = (V, E)$ consists of V, a nonempty set of *vertices* (or *nodes*), and E, a set of *directed edges* or *arcs*. Each edge is an ordered pair of vertices. The directed edge (u,v) is said to start at u and end at v.

Definition: Let (u,v) be an edge in G. Then u is the *initial vertex* of this edge and is *adjacent to* v and v is the *terminal* (or *end*) *vertex* of this edge and is *adjacent from* u. The initial and terminal vertices of a loop are the same.

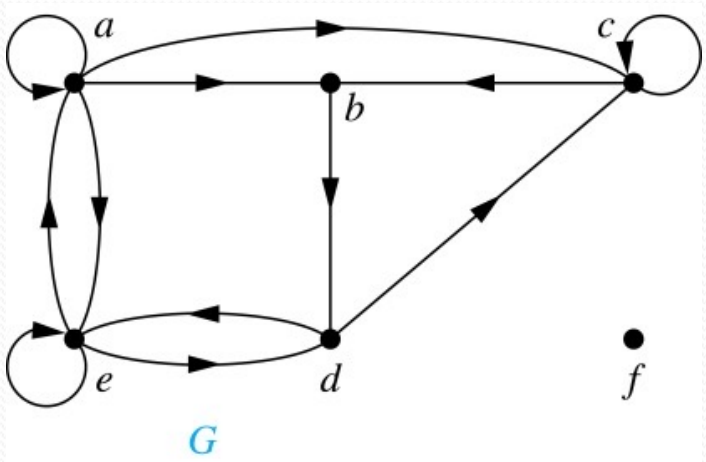
定义 当(u,v)是带有有向边的图G的边时，说u邻接到v，而且说v从u邻接。顶点u称为(u,v)的起点，v称为(u,v)的终点。环的起点和终点是相同的。

Directed Graphs (*continued*)

- **Definition:** The *in-degree of a vertex v*, denoted $deg^-(v)$, is the number of edges which terminate at *v*. The *out-degree of v*, denoted $deg^+(v)$, is the number of edges with *v* as their initial vertex. Note that a loop at a vertex contributes 1 to both the in-degree and the out-degree of the vertex.
- 定义：在带有向边的图里，顶点***v***的入度$\boldsymbol{deg^-(v)}$是以***v***作为终点的边数。顶点***v***的出度$\boldsymbol{deg^+(v)}$是以***v***作为起点的边数。注意顶点上的环对这个顶点的入度和出度的贡献都是**1.**

Directed Graphs (*continued*)

Example: In the graph G we have



所有顶点的入度总和？
所有顶点的出度总和？



$\deg^-(a) = 2, \deg^-(b) = 2, \deg^-(c) = 3, \deg^-(d) = 2,$
$\deg^-(e) = 3, \deg^-(f) = 0.$

$\deg^+(a) = 4, \deg^+(b) = 1, \deg^+(c) = 2, \deg^+(d) = 2,$
$\deg^+(e) = 3, \deg^+(f) = 0.$

Directed Graphs (*continued*)

Theorem 3: Let $G = (V, E)$ be a graph with directed edges. Then:

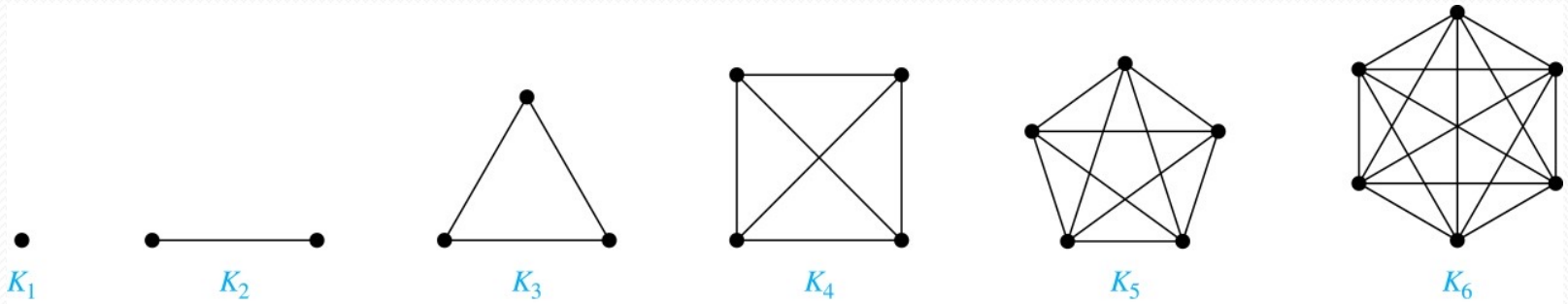
$$|E| = \sum_{v \in V} deg^-(v) = \sum_{v \in V} deg^+(v).$$

Proof: The first sum counts the number of outgoing edges over all vertices and the second sum counts the number of incoming edges over all vertices. It follows that both sums equal the number of edges in the graph. ◀

Special Types of Simple Graphs: Complete Graphs 完全图

A *complete graph on n vertices*, denoted by K_n, is the simple graph that contains exactly one edge between each pair of distinct vertices.

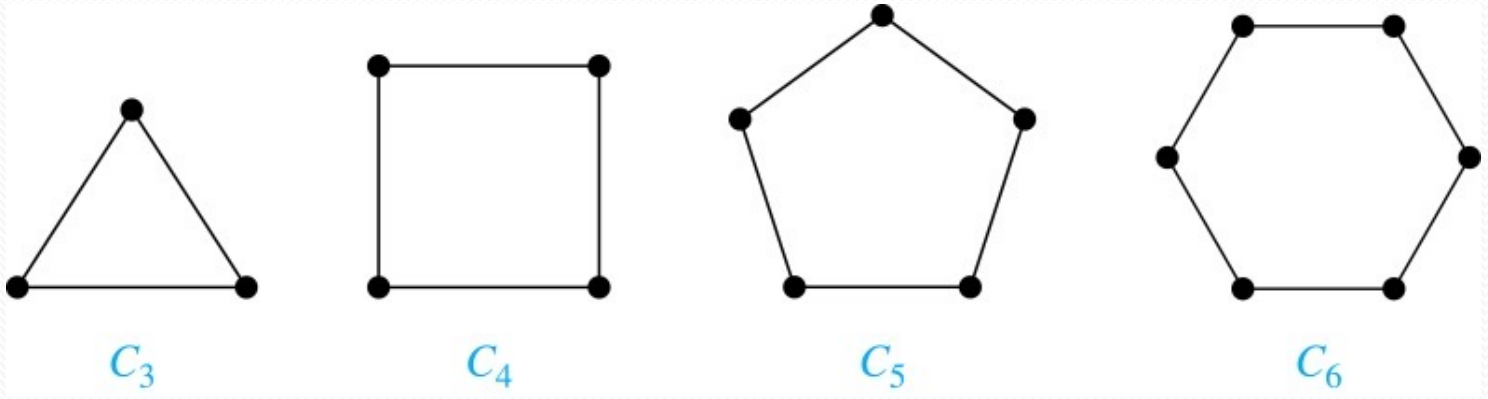
*n*个顶点的**完全图**（表示为K_n）是在每对不同顶点之间都恰有一条边的简单图。



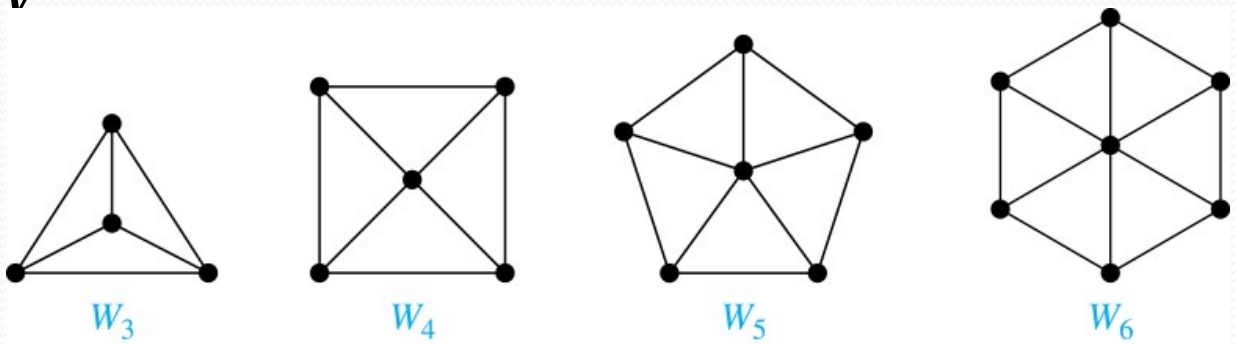
Special Types of Simple Graphs: Cycles and Wheels

A *cycle* C_n for $n \geq 3$ consists of n vertices $v_1, v_2, \cdots, v_n$, and edges $\{v_1, v_2\}, \{v_2, v_3\}, \cdots, \{v_{n-1}, v_n\}, \{v_n, v_1\}$.

圈图 C_n是由n个顶点$v_1, v_2, \cdots, v_n$以及边$\{v_1, v_2\}, \{v_2, v_3\}, \cdots, \{v_{n-1}, v_n\}, \{v_n, v_1\}$组成的。



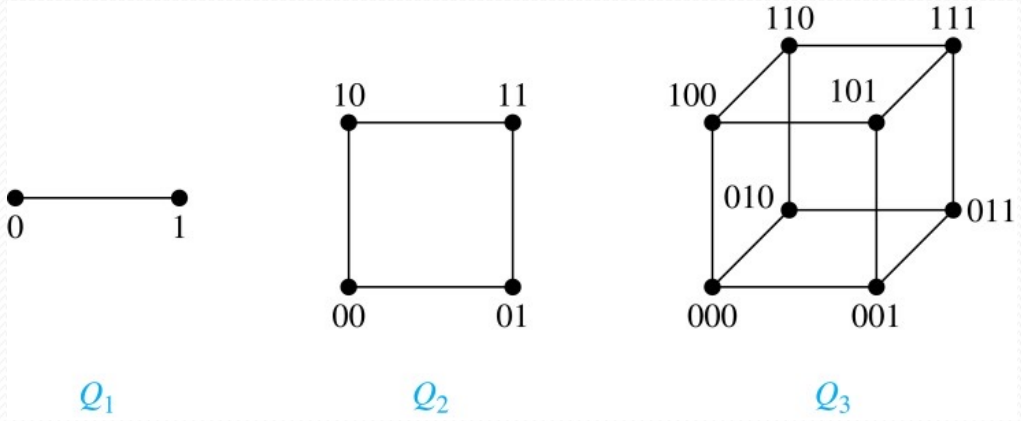
- A *wheel* W_n is obtained by adding an additional vertex to a cycle C_n for $n \geq 3$ and connecting this new vertex to each of the n vertices in C_n by new edges.
- **轮图** 对$n \geq 3$来说，当给圈图C_n添加另一个顶点，而且把这个新顶点与C_n里n个顶点逐个连接时，就得出轮图W



Special Types of Simple Graphs: *n*-Cubes

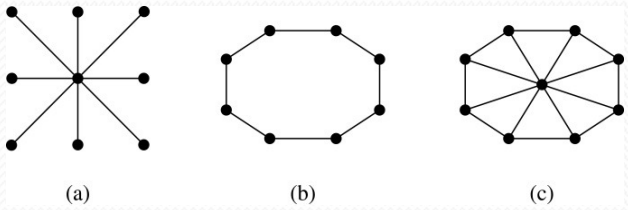
An *n-dimensional hypercube*, or *n-cube*, Q_n, is a graph with 2^n vertices representing all bit strings of length n, where there is an edge between two vertices that differ in exactly one bit position.

n立方体图Q_n 是用顶点表示2^n 个长度为n的位串的图。两个顶点相邻，当且仅当他们所表示的位串恰恰相差一位。

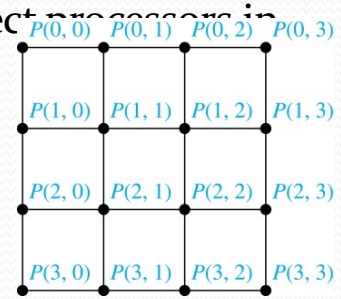


Special Types of Graphs and Computer Network Architecture

Various special graphs play an important role in the design of computer networks. 局域网



- Some local area networks use a *star topology* 星型拓扑, which is a complete bipartite graph $K_{1,n}$, as shown in (a). All devices are connected to a central control device.
- Other local networks are based on a *ring topology* 环形拓扑, where each device is connected to exactly two others using C_n, as illustrated in (b). Messages may be sent around the ring.
- Others, as illustrated in (c), use a W_n – based topology, combining the features of a star topology and a ring topology 混合形式.
- Various special graphs also play a role in parallel processing where processors need to be interconnected as one processor may need the output generated by another.
  - The *n-dimensional hypercube*, or *n-cube*, Q_n, is a common way to connect processors in parallel, e.g., Intel Hypercube.
  - Another common method is the *mesh* network, illustrated here for 16 processors.



Bipartite Graphs 偶图、二分（部）图

Definition: A simple graph G is bipartite if V can be partitioned into two disjoint subsets V_1 and V_2 such that every edge connects a vertex in V_1 and a vertex in V_2. In other words, there are no edges which connect two vertices in V_1 or in V_2.

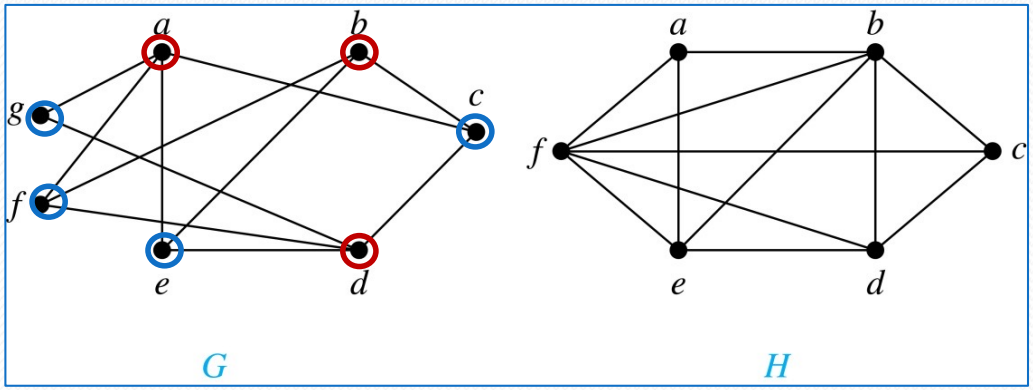
定义：若把简单图G的顶点集分成两个不相交的非空集合 V_1 和 V_2，使得图里的每一条边都连接着V_1 里的一个顶点与V_2里的一个顶点（因此G里没有边是连接着V_1 里的两个顶点或V_2里的两个顶点），则G称为**偶图**。当此条件成立，称(V_1, V_2)为G的顶点集的一个二部划分。

Bipartite Graphs 偶图、二分（部）图

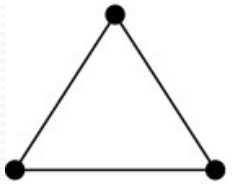
定理4 一个简单图是偶图，当且仅当能够对图中的每个顶点赋以两种不同的颜色，而不让相邻接的顶点被赋以相同的颜色。

- 该定理给出了决定图是否为偶图的有用准则。

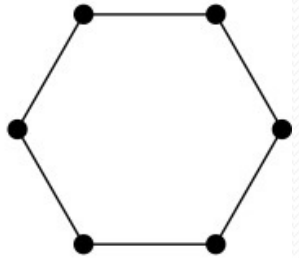
G is bipartite



H is not bipartite since if we color *a* red, then the adjacent vertices *f* and *b* must both be blue.



C_3



C_6

圈图C_3、C_6是偶图吗？

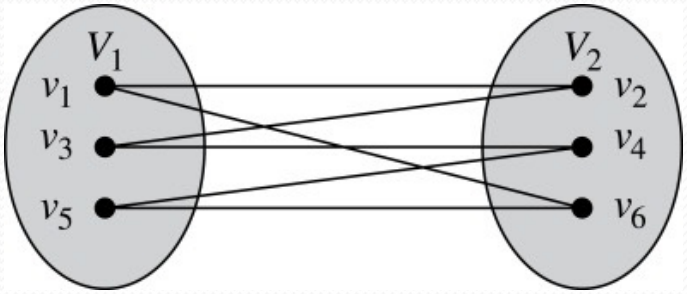
Solution: C_3不是偶图。If we divide the vertex set of C_3 into two nonempty sets, one of the two must contain two vertices. But in C_3 every vertex is connected to every other vertex. Therefore, the two vertices in the same partition are connected. Hence, C_3 is not bipartite.

Solution: C_6是偶图

We can partition the vertex set into $V_1 = \{v_1, v_3, v_5\}$ and $V_2 = \{v_2, v_4, v_6\}$ so that every edge of C_6 connects a vertex in V_1 and V_2 .

结论：对圈图C_n而言，当n是偶数时，C_n是偶图。

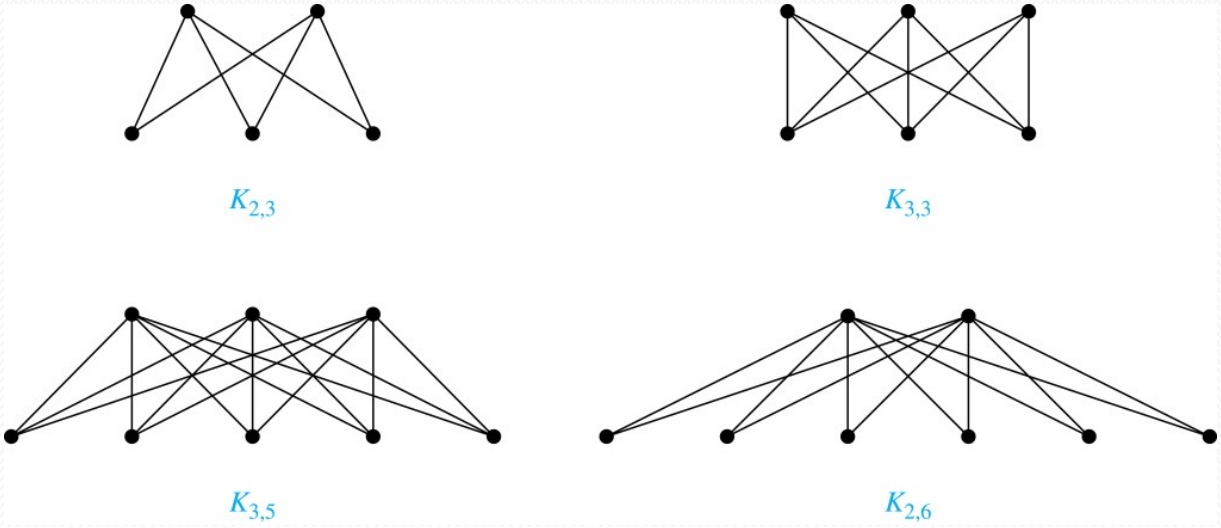
当n是奇数时，C_n不是偶图。



Complete Bipartite Graphs

Definition: A *complete bipartite graph* $K_{m,n}$ is a graph that has its vertex set partitioned into two subsets V_1 of size m and V_2 of size n such that there is an edge from every vertex in V_1 to every vertex in V_2.

定义：完全偶图 $K_{m,n}$是顶点集分成分别含有m和n个顶点的两个子集的图。两个顶点之间有边当且仅当一个顶点属于第一个子集而另一个顶点属于第二个子集。

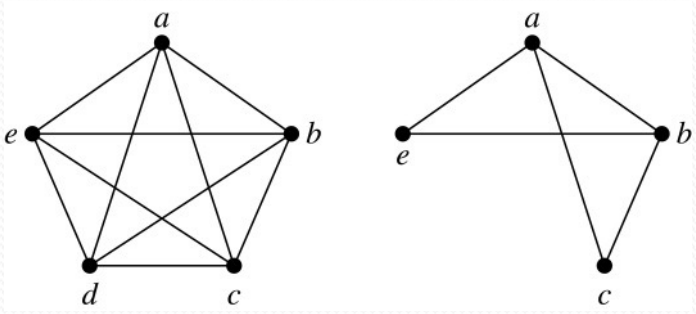


New Graphs from Old

Definition: A *subgraph of a graph* $G = (V,E)$ is a graph (W,F), where $W \subset V$ and $F \subset E$. A subgraph H of G is a proper subgraph of G if $H \neq G$.

定义：图$G = (V,E)$的子图是图$H=(W,F)$，其中$W \subset V$且$F \subset E$.

Example: Here we show K_5 and one of its subgraphs.

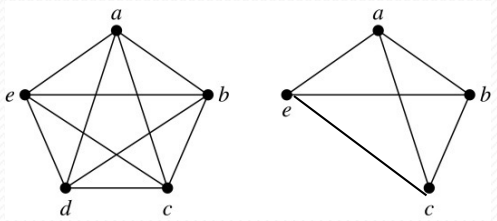


New Graphs from Old

Definition: Let $G = (V, E)$ be a simple graph. The *subgraph induced* by a subset W of the vertex set V is the graph (W,F), where the edge set F contains an edge in E if and only if both endpoints are in W.

定义：如果W是V的一个子集，以两个顶点均在W中的边集组成的图称为图G的**顶点导出子图**。

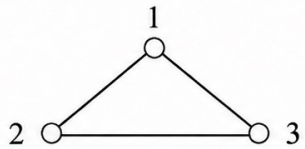
Example: Here we show K_5 and the subgraph induced by $W = \{a,b,c,e\}$.



New Graphs from Old

- 定义：如果图$G = (V, E)$ 的一个子图$H=(W,F)$包含G的所有顶点$V\,(W = V)$，称为该子图为G的一个生成子图。

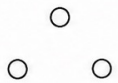
如图所示，求G的所有生成子图



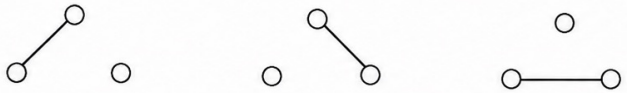
图G

解：按边数分别求出

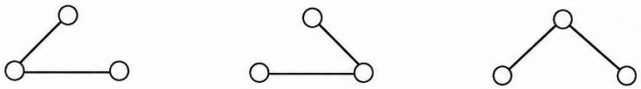
0条边：



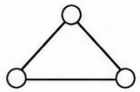
1条边：



2条边：

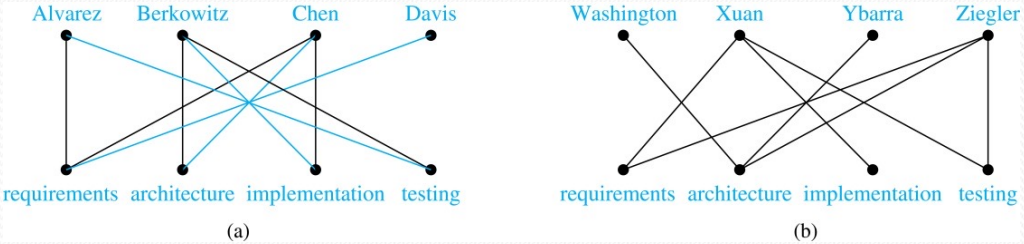


3条边：



Bipartite Graphs and Matchings

- Bipartite graphs are used to model applications that involve matching the elements of one set to elements in another, for example:
- *Job assignments* - vertices represent the jobs and the employees, edges link employees with those jobs they have been trained to do. A common goal is to match jobs to employees so that the most jobs are done.



- *Marriage* - vertices represent the men and the women and edges link a a man and a woman if they are an acceptable spouse. We may wish to find the largest number of possible marriages.

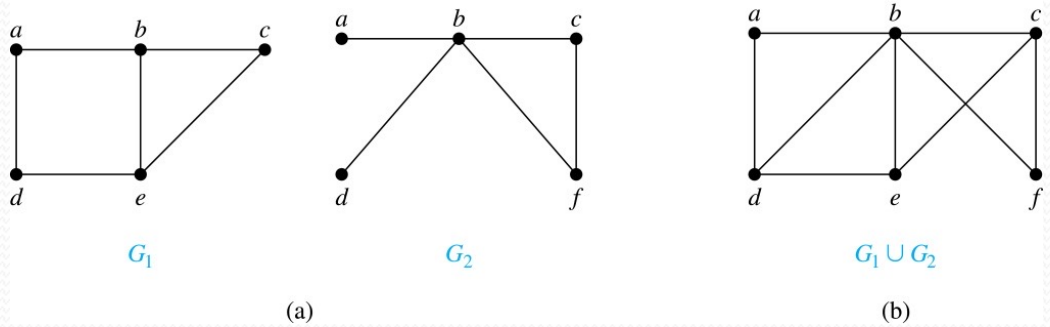
See the text for more about matchings in bipartite graphs.

New Graphs from Old (*continued*)

Definition: The *union* of two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is the simple graph with vertex set $V_1 \cup V_2$ and edge set $E_1 \cup E_2$. The union of G_1 and G_2 is denoted by $G_1 \cup G_2$.

定义：两个简单图$G_1 = (V_1, E_1)$ 和 $G_2 = (V_2, E_2)$ 的并图是带有顶点集$V_1 \cup V_2$和边集$E_1 \cup E_2$的简单图。

Example:



Representing Graphs and Graph Isomorphism
图的表示和图的同构

Section Summary

- Adjacency Lists
- Adjacency Matrices
- Incidence Matrices
- Isomorphism of Graphs

Representing Graphs: Adjacency Lists

Definition: An *adjacency list* can be used to represent a graph with no multiple edges by specifying the vertices that are adjacent to each vertex of the graph.

定义：邻接表通过列出与图的每个顶点相邻的顶点来表示不带多重边的图的方式。

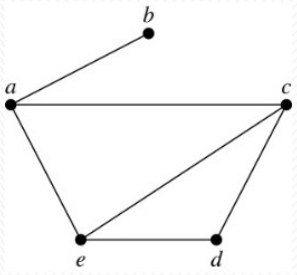


TABLE 1 An Adjacency List for a Simple Graph.

Vertex	*Adjacent Vertices*
a	b, c, e
b	a
c	a, d, e
d	c, e
e	a, c, d

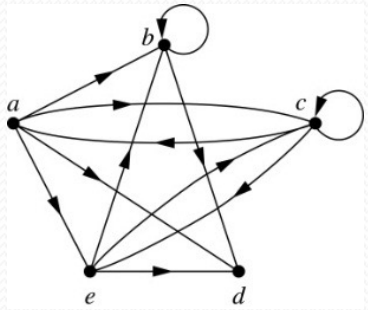


TABLE 2 An Adjacency List for a Directed Graph.

Initial Vertex	*Terminal Vertices*
a	b, c, d, e
b	b, d
c	a, c, e
d	
e	b, c, d

Representation of Graphs: Adjacency Matrices

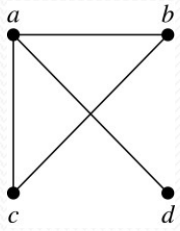
Definition: Suppose that $G = (V, E)$ is a simple graph where $|V| = n$. Arbitrarily list the vertices of G as $v_1, v_2, \ldots, v_n$. The *adjacency matrix* $\mathbf{A}_G$ of G, with respect to the listing of vertices, is the $n \times n$ zero-one matrix with 1 as its (i, j)th entry when v_i and v_j are adjacent, and 0 as its (i, j)th entry when they are not adjacent.

定义：假设 $G = (V, E)$ 是简单图，$|V| = n$。把G的顶点任意地排列成$v_1, v_2, \ldots, v_n$。对这个顶点表来说，G的邻接矩阵$\mathbf{A}_G$是一个$n \times n$的0-1矩阵，它满足的性质为：当v_i和v_j相邻时第(i, j)项是1，当v_i和v_j不相邻时第(i, j)项是0。

$$\mathbf{A}_G = [a_{ij}], \quad a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

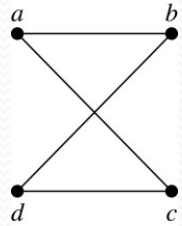
Adjacency Matrices (*continued*)

Example:



$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.



$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

The ordering of vertices is a, b, c, d.

When a graph is sparse, that is, it has few edges relatively to the total number of possible edges, it is much more efficient to represent the graph using an adjacency list than an adjacency matrix. But for a dense graph, which includes a high percentage of possible edges, an adjacency matrix is preferable.

Note: The adjacency matrix of a simple graph is symmetric, i.e., $a_{ij} = a_{ji}$
Also, since there are no loops, each diagonal entry a_{ij} for $i = 1, 2, 3, \ldots, n$, is 0.

Adjacency Matrices (*continued*)

- Adjacency matrices can also be used to represent graphs with loops and multiple edges.
- A loop at the vertex v_i is represented by a 1 at the (i, i)th position of the matrix.
- When multiple edges connect the same pair of vertices v_i and v_j, (or if multiple loops are present at the same vertex), the (i, j)th entry equals the number of edges connecting the pair of vertices.

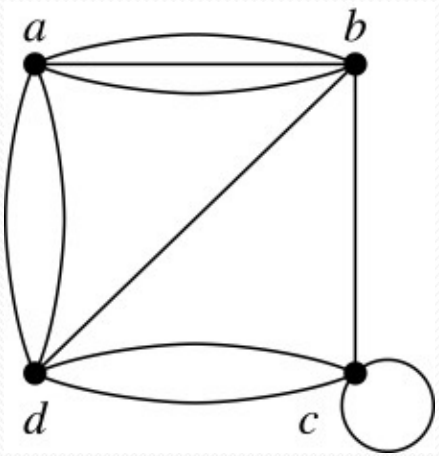
- 邻接矩阵可以表示带环和多重边的无向图。

- 顶点v_i 上的环表示成邻接矩阵的第(i, i)位置上的1。

- 当出现多重边时，邻接矩阵不再是0-1矩阵，第(i, j)项等于与(v_i, v_j)关联的边数。

Adjacency Matrices (*continued*)

- **Example**: We give the adjacency matrix of the pseudograph shown here using the ordering of vertices a, b, c, d.
- 顶点顺序为a, b, c, d的邻接矩阵是



$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$

Adjacency Matrices (*continued*)

- Adjacency matrices can also be used to represent directed graphs. The matrix for a directed graph $G = (V, E)$ has a 1 in its (i, j)th position if there is an edge from v_i to v_j, where $v_1, v_2, \ldots v_n$ is a list of the vertices.
- 邻接矩阵可以表示有向图，若 v_i 到 v_j有边，则矩阵在(i, j)位置上是1，其中$v_1, v_2, \ldots v_n$ 是有向图任意的顶点序列。
  - In other words, if the graphs adjacency matrix is $\mathbf{A}_G = [a_{ij}]$, then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

- The adjacency matrix for a directed graph does not have to be symmetric, because there may not be an edge from v_i to v_j, when there is an edge from v_j to v_i.
- 有向图的邻接矩阵不必是对称的，因为当从v_i 到v_j 有边时，从v_j 到v_i 可以没有边。

- To represent directed multigraphs, the value of a_{ij} is the number of edges connecting v_i to v_j.
- 表示有向多重边时，a_{ij}等于关联到(v_i,v_j) 的边数。

Representation of Graphs: Incidence Matrices 关联矩阵

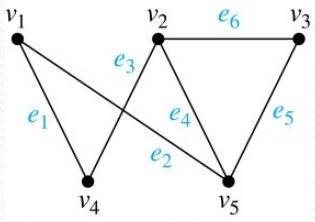
Definition: Let $G = (V, E)$ be an undirected graph with vertices where $v_1, v_2, \ldots v_n$ and edges $e_1, e_2, \ldots e_m$. The incidence matrix with respect to the ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where

$G = (V, E)$是无向图，$v_1, v_2, \ldots v_n$ 是顶点而$e_1, e_2, \ldots e_m$是边。则相对于V和E的这个顺序的关联矩阵是$n \times m$矩阵$\mathbf{M} = [m_{ij}]$,其中

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

Incidence Matrices (*continued*)

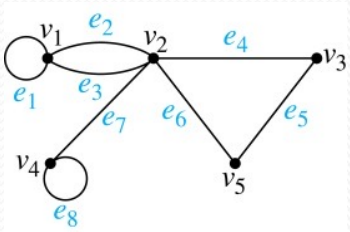
Example: Simple Graph and Incidence Matrix



$$\begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

The rows going from top to bottom represent v_1 through v_5 and the columns going from left to right represent e_1 through e_6.

Example: Pseudograph and Incidence Matrix



$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

The rows going from top to bottom represent v_1 through v_5 and the columns going from left to right represent e_1 through e_8.

Isomorphism of Graphs图的同构

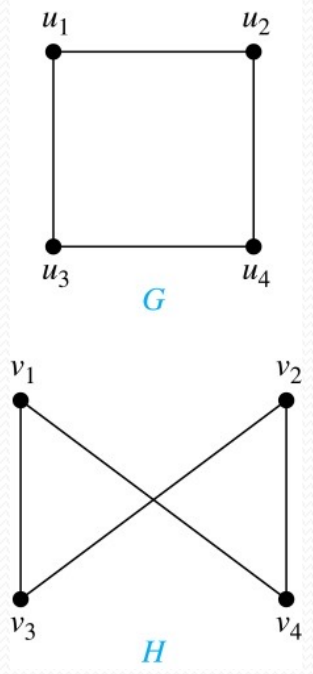
Definition: The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there is a one-to-one and onto function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2, for all a and b in V_1. Such a function f is called an *isomorphism*. Two simple graphs that are not isomorphic are called *nonisomorphic*.

定义：设$G_1 = (V_1, E_1)$和$G_2 = (V_2, E_2)$是简单图，若存在一对一的和映上的从V_1到V_2的函数f，且f具有这样的性质：对V_1里的所有a和b来说，a和b在G_1里相邻当且仅当$f(a)$和$f(b)$在G_2里相邻，就说G_1和G_2是同构的。这样的函数f称为同构。

Isomorphism of Graphs (*cont.*)

Example: Show that the graphs $G = (V, E)$ and $H = (W, F)$ are isomorphic.

证明图***G***和***H***是同构的。



Solution: The function f with $f(u_1) = v_1$, $f(u_2) = v_4$, $f(u_3) = v_3$, and $f(u_4) = v_2$ is a one-to-one correspondence between V and W. Note that adjacent vertices in G are u_1 and u_2, u_1 and u_3, u_2 and u_4, and u_3 and u_4. Each of the pairs $f(u_1) = v_1$ and $f(u_2) = v_4$, $f(u_1) = v_1$ and $f(u_3) = v_3$, $f(u_2) = v_4$ and $f(u_4) = v_2$, and $f(u_3) = v_3$ and $f(u_4) = v_2$ consists of two adjacent vertices in H.

Isomorphism of Graphs (*cont.*)

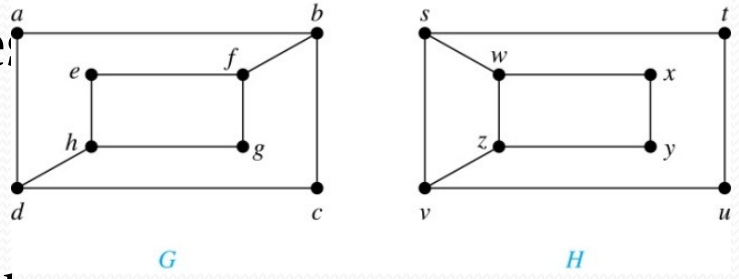
- It is difficult to determine whether two simple graphs are isomorphic using brute force because there are $n!$ possible one-to-one correspondences between the vertex sets of two simple graphs with n vertices.
- The best algorithms for determining weather two graphs are isomorphic have exponential worst case complexity in terms of the number of vertices of the graphs.
- 判定两个简单图是否同构常常是一件困难的事情。在两个带有n个顶点的简单图顶点集之间有$n!$种可能的一一对应。若n太大，通过检验每一种对应来看是否保持相邻关系是不可行的。
- 判定两个图是否同构，已知的最好算法具有指数的最坏情形时间复杂性（相对于图的顶点数而言）。

Isomorphism of Graphs (*cont.*)

- Sometimes it is not hard to show that two graphs are not isomorphic. We can do so by finding a property, preserved by isomorphism, that only one of the two graphs has. Such a property is called *graph invariant*.
- There are many different useful graph invariants that can be used to distinguish nonisomorphic graphs, such as the number of vertices, number of edges, and degree sequence (list of the degrees of the vertices in nonincreasing order). We will encounter others in later sections of this chapter.
- 有时候说明两个图不同构并不困难，只要能找到某个属性，两个图中只有一个图具有这个属性，且该属性只在讨论同构时使用，就可以说这两个图不同构。这样的属性叫做同构的不变量。
- 同构的不变量有很多，比如顶点数、边数以及顶点度等。若两个简单图中的这些量有任何不同，则这两个图就不是同构的。但是，当这些量都相同时，也不一定意味着两个图同构。

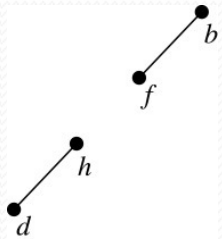
Isomorphism of Graphs (*cont.*)

Example: Determine whether the two graphs are isomorphic.
判定两个图是否同构。

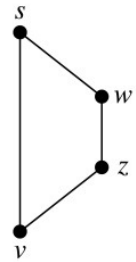


Solution: Both graphs have eight vertices and ten edges.
They also both have four vertices of degree two and four of degree three.

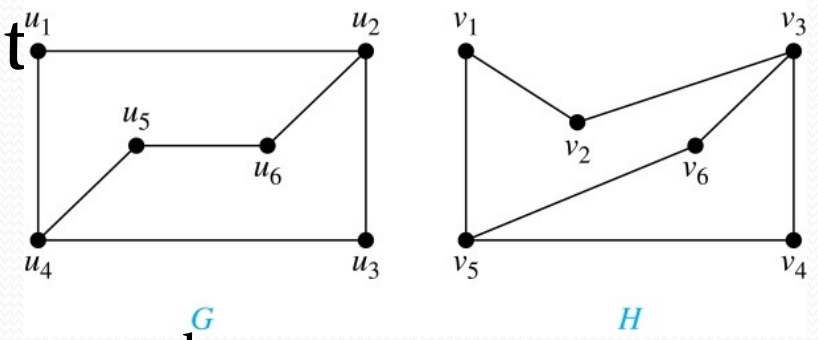
However, G and H are not isomorphic. Note that since $deg(a) = 2$ in G, a must correspond to t, u, x, or y in H, because these are the vertices of degree 2. But each of these vertices is adjacent to another vertex of degree two in H, which is not true for a in G.



Alternatively, note that the subgraphs of G and H made up of vertices of degree three and the edges connecting them must be isomorphic.
But the subgraphs, as shown at the right, are not isomorphic.



Example: Determine whether t
two graphs are isomorphic.
判定两个图是否同构。



Solution: Both graphs have six vertices and seven edges.
They also both have four vertices of degree two and two of degree three.
The subgraphs of G and H consisting of all the vertices of degree two and the edges connecting them are isomorphic. So, it is reasonable to try to find an isomorphism f.
We define an injection f from the vertices of G to the vertices of H that preserves the degree of vertices. We will determine whether it is an isomorphism.

The function f with $f(u_1) = v_6$, $f(u_2) = v_3$, $f(u_3) = v_4$, and $f(u_4) = v_5$, $f(u_5) = v_1$, and $f(u_6) = v_2$ is a one-to-one correspondence between G and H. Showing that this correspondence preserves edges is straightforward, so we will omit the details here. Because f is an isomorphism, it follows that G and H are isomorphic graphs.

Algorithms for Graph Isomorphism

- The best algorithms known for determining whether two graphs are isomorphic have exponential worst-case time complexity (in the number of vertices of the graphs).
- However, there are algorithms with linear average-case time complexity.
- You can use a public domain program called NAUTY to determine in less than a second whether two graphs with as many as 100 vertices are isomoprhic.
- Graph isomorphism is a problem of special interest because it is one of a few NP problems not known to be either tractable or NP-complete.
- 判定两个图是否同构，已知的最好算法具有指数的最坏情形时间复杂性（相对于图的顶点数而言）。
- 解决这个问题的线性平均情形时间复杂性的算法已经找到。
- 名为NAUTY 的最佳实用算法可在1秒钟之内判定带有100个顶点的两个图是否同构。
- 图的同构问题是一个NP难问题。

Applications of Graph Isomorphism

- The question whether graphs are isomorphic plays an important role in applications of graph theory. For example,
  - chemists use molecular graphs to model chemical compounds. Vertices represent atoms and edges represent chemical bonds. When a new compound is synthesized, a database of molecular graphs is checked to determine whether the graph representing the new compound is isomorphic to the graph of a compound that this already known.
  - Electronic circuits are modeled as graphs in which the vertices represent components and the edges represent connections between them. Graph isomorphism is the basis for
    - the verification that a particular layout of a circuit corresponds to the design's original schematics.
    - determining whether a chip from one vendor includes the intellectual property of another vendor.

Connectivity
连通性

Section Summary

- Paths
- Connectedness in Undirected Graphs
- Vertex Connectivity and Edge Connectivity (*not currently included in overheads*)
- Connectedness in Directed Graphs
- Paths and Isomorphism (*not currently included in overheads*)
- Counting Paths between Vertices

Paths 通路

Informal Definition: A *path* is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex along edges of the graph. As the path travels along its edges, it visits the vertices along this path, that is, the endpoints of these.

非形式化定义：**通路**是边的序列，它从图的一个顶点上开始并且经过图的边，总是连接相邻的顶点对。

Applications: Numerous problems can be modeled with paths formed by traveling along edges of graphs such as:

- determining whether a message can be sent between two computers.
- efficiently planning routes for mail delivery.

通路

定义：设n是非负整数且G是**无向图**。在G中从u到v的长度为n的通路是G的n条边$e_1, \dots, e_n$的序列，使得f(e_1)={x_0, x_1},f(e_2)={x_1,x_2},...,f(e_n)={x_{n-1}, x_n}，其中$x_0 = u$而$x_n = v$。

- ✓ 当这个图是简单图时，就用顶点序列$x_0, x_1, \dots, x_n$表示这条通路（因为列出这些顶点就唯一地确定了通路）。
- ✓ 若一条通路在相同的顶点上开始和结束，即$u = v$且长度大于0，则它是一条回路。
- ✓ 把这条通路或回路说成是经过顶点$x_1, x_2, \dots, x_{n-1}$或遍历边$e_1, \dots, e_n$。

若通路或回路不重复地包含相同的边，则它是简单的。

Paths

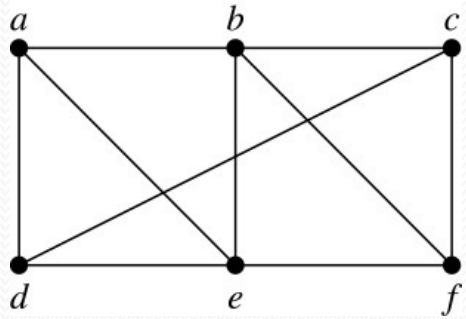
Definition: Let n be a nonnegative integer and G an undirected graph. A *path* of *length* n from u to v in G is a sequence of n edges $e_1, \ldots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \ldots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \ldots, n$, the endpoints x_{i-1} and x_i.

- When the graph is simple, we denote this path by its vertex sequence $x_0, x_1, \ldots, x_n$ (since listing the vertices uniquely determines the path).
- The path is a *circuit* if it begins and ends at the same vertex ($u = v$) and has length greater than zero.
- The path or circuit is said to *pass through* the vertices $x_1, x_2, \ldots, x_{n-1}$ and *traverse* the edges $e_1, \ldots, e_n$.
- A path or circuit is *simple* if it does not contain the same edge more than once.

This terminology is readily extended to directed graphs. (*see text*)

Paths (*continued*)

Example: In the simple graph here:



a, *d*, *c*, *f*, *e* is a simple path of length 4.
d, *e*, *c*, *a* is not a path because *e* is not connected to *c*.
b, *c*, *f*, *e*, *b* is a circuit of length 4.
a, *b*, *e*, *d*, *a*, *b* is a path of length 5, but it is not a simple path.

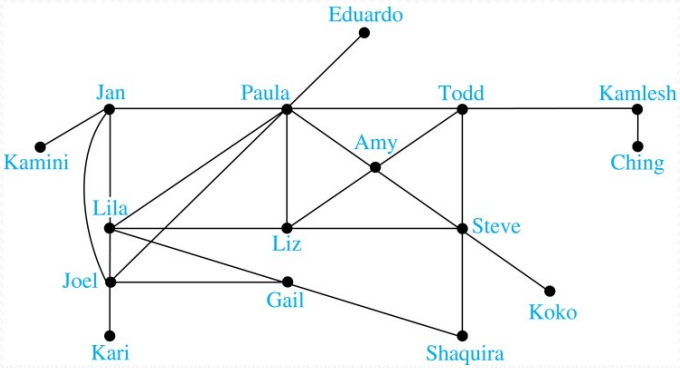
通路

定义：设n是非负整数且G是**有向多重图**。在G中从u到v的长度为n的通路是G的边$e_1, \ldots, e_n$的序列，使得$f(e_1)=\{x_0, x_1\}, f(e_2)=\{x_1, x_2\}, \ldots, f(e_n)=\{x_{n-1}, x_n\}$，其中$x_0 = u$而$x_n = v$。

- ✓ 当有向图里没有多重边时，就用顶点序列$x_0, x_1, \ldots, x_n$表示这条通路。
- ✓ 把在相同顶点上开始和结束的长度大于0的通路称为回路或圈。
- ✓ 若一条通路或回路不重复地包含相同的边，则它是简单的。

Degrees of Separation

Example: *Paths in Acquaintanceship Graphs*. In an acquaintanceship graph there is a path between two people if there is a chain of people linking these people, where two people adjacent in the chain know one another. In this graph there is a chain of six people linking Kamini and Ching.



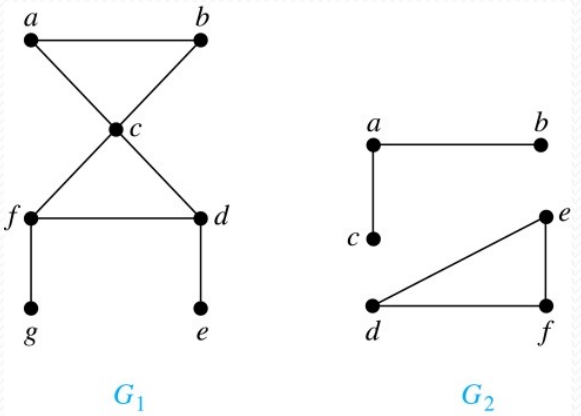
Some have speculated that almost every pair of people in the world are linked by a small chain of no more than six, or maybe even, five people. The play *Six Degrees of Separation* by John Guare is based on this notion.

Connectedness in Undirected Graphs
无向图的连通性

Definition: An undirected graph is called *connected* if there is a path between every pair of vertices. An undirected graph that is not *connected* is called *disconnected*. We say that we *disconnect* a graph when we remove vertices or edges, or both, to produce a disconnected subgraph.

定义：若无向图每一对不同的顶点之间都有通路，则称该图为连通的。

Example: G_1 is connected because there is a path between any pair of its vertices, as can be easily seen. However G_2 is not connected because there is no path between vertices *a* and *f*, for example.



Connectedness in Undirected Graphs 无向图的连通性

定理：在连通无向图的每一对不同顶点之间都存在简单通路。

证明：设u和v是连通无向图G的两个不同的顶点。因为G是连通的，所以u和v之间至少有1条通路。设x_0，x_1, ..., x_n是长度最短的通路的顶点序列，其中x_0=u，x_n=v。这条长度最短的通路是简单的。

假设不是简单的，则对满足0≤i≤j, 的某个i和j来说，有x_i=x_j, 那么可以通过删除顶点序列x_i, ..., x_{j-1}，所对应的边，就获得了带有顶点序列x_0，x_1, ..., x_{i-1}, x_j, ..., x_n的从u到v的更短的通路。

Connected Components连通分支

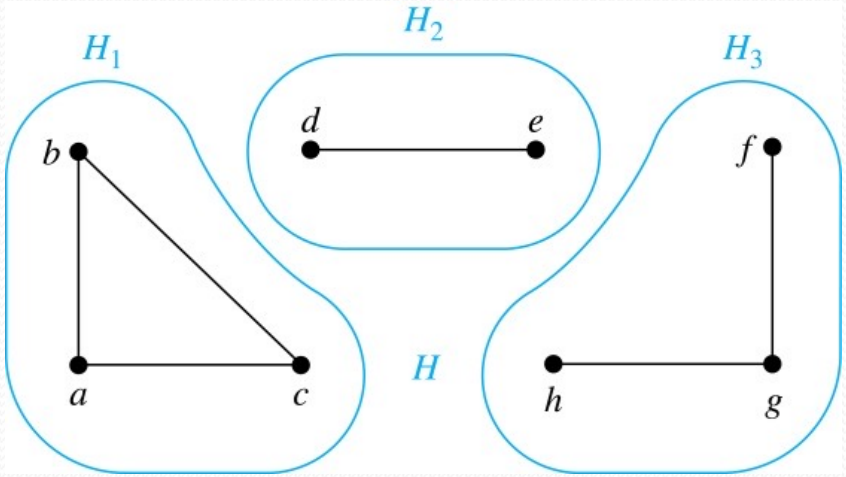
Definition: A *connected component* of a graph *G* is a connected subgraph of *G* that is not a proper subgraph of another connected subgraph of *G*. A graph *G* that is not connected has two or more connected components that are disjoint and have *G* as their union.

定义：不连通的图是2个或2个以上连通子图的并图，每一对子图都没有公共的顶点，这些不相交的连通子图称为**图的连通分支**。

Connected Components连通分支

Example: The graph H is the union of three disjoint subgraphs H_1, H_2, and H_3, none of which are proper subgraphs of a larger connected subgraph of G.These three subgraphs are the connected components of H.

图H的连通分支：3个不相交连通子图H_1, H_2, 和 H_3。



Connectedness in Directed Graphs
有向图的连通性

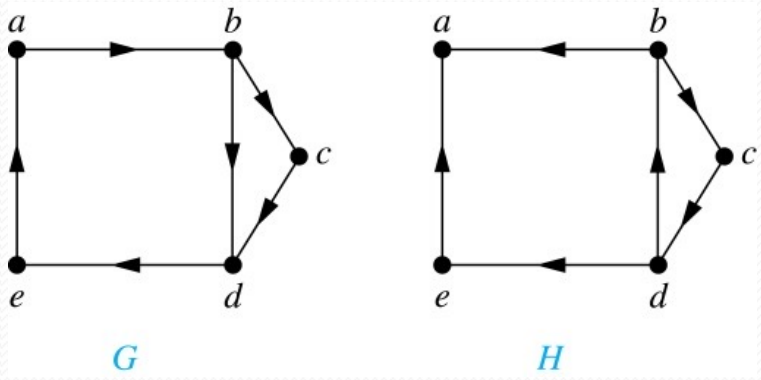
Definition: A directed graph is *strongly connected* if there is a path from *a* to *b* and a path from *b* to *a* whenever *a* and *b* are vertices in the graph.

定义：若每当*a*和*b*都是一个有向图里的顶点时，就有从*a*到*b*和从*b*到*a*的通路，则该图是**强连通**的。

Definition: A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph, which is the undirected graph obtained by ignoring the directions of the edges of the directed graph.

定义：若一个有向图在其对应的无向图中任意两个顶点之间都存在一条路径，则称该有向图是**弱连通的**。有向图是弱连通的，当且仅当在忽略边的方向时，任何两个顶点之间总是存在通路。显然，任何强联通有向图都是弱连通的。

下图的有向图G和H是否为强连通的？是否为弱连通？



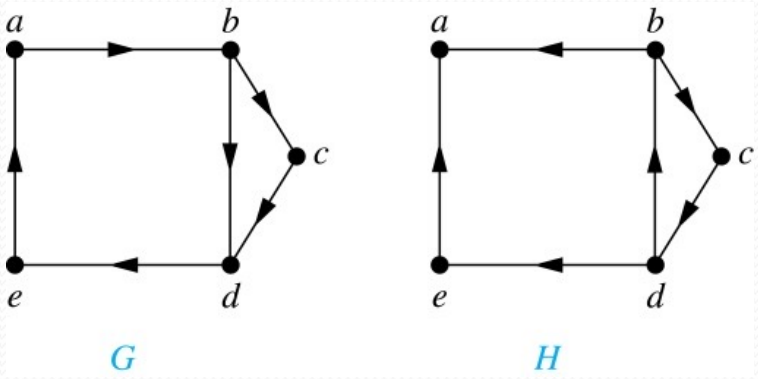
- **Solution**: *G* is strongly connected because there is a path between any two vertices in the directed graph.
  Hence, *G* is also weakly connected.
  The graph *H* is not strongly connected, since there is no directed path from *a* to *b*, but it is weakly connected.

Connectedness in Directed Graphs

Definition: The subgraphs of a directed graph *G* that are strongly connected but not contained in larger strongly connected subgraphs, that is, the maximal strongly connected subgraphs, are called the *strongly connected components* or *strong components* of *G*.

定义：有向图G的子图是强连通的而不包含在更大的强连通子图中，即极大强连通子图，可称之为**G的强连通分支或强分支。**

下图的有向图H中有哪些强连通分支？



- **Solution**: The graph *H* has three strongly connected components, consisting of the vertex *a*; the vertex *e*; and the subgraph consisting of the vertices *b*, *c*, *d* and edges (*b*,*c*), (*c*,*d*), and (*d*,*b*).

Counting Paths between Vertices
计算顶点之间的通路数

- We can use the adjacency matrix of a graph to find the number of paths between two vertices in the graph.
- 在一个图里两个顶点之间的通路的数目，可以用这个图的邻接矩阵来确定。

Theorem: Let G be a graph with adjacency matrix **A** with respect to the ordering $v_1, \ldots, v_n$ of vertices (with directed or undirected edges, multiple edges and loops allowed). The number of different paths of length r from v_i to v_j, where $r > 0$ is a positive integer, equals the (i,j)th entry of $\mathbf{A}^r$.

定理：设G是带有相对于顶点顺序 $v_1, \ldots, v_n$的邻接矩阵A的图（允许带有无向或有向边、带有多重边和环）。从v_i 到 v_j的长度为r的不同通路的数目等于 $\mathbf{A}^r$的第 (i,j)项，其中r是正整数。

数学归纳法证明：

基础步骤： 根据邻接矩阵的定义，从 v_i 到 v_j 的长度为 1 的路径数，正好等于矩阵 A 的第 (i, j) 个元素。

归纳步骤： 归纳假设为：矩阵 A^r 的第 (i, j) 个元素等于从 v_i 到 v_j 的长度为 r 的不同路径数。

由于

$$A^{r+1} = A^r A,$$

因此 A^{r+1} 的第 (i, j) 个元素等于

$$b_{i1}a_{1j} + b_{i2}a_{2j} + \cdots + b_{in}a_{nj},$$

其中 b_{ik} 是矩阵 A^r 的第 (i, k) 个元素。由归纳假设可知，b_{ik} 表示从 v_i 到 v_k 的长度为 r 的路径数。

一条从 v_i 到 v_j、长度为 $r + 1$ 的路径，可以看作由两部分组成：先是一条从 v_i 到某个顶点 v_k 的长度为 r 的路径，然后再接上一条从 v_k 到 v_j 的边。根据计数中的乘法原理，这类路径的数量等于从 v_i 到 v_k 的长度为 r 的路径数，即 b_{ik}，乘以从 v_k 到 v_j 的边数，即 a_{kj}。

对所有可能的中间顶点 v_k 求和，得到总路径数为

$$b_{i1}a_{1j} + b_{i2}a_{2j} + \cdots + b_{in}a_{nj}.$$

这正是 A^{r+1} 的第 (i, j) 个元素。因此，命题对 $r + 1$ 成立。由数学归纳法可知，对所有正整数 r，矩阵 A^r 的第 (i, j) 个元素都等于从 v_i 到 v_j 的长度为 r 的路径数。

Proof by mathematical induction:

Basis Step: By definition of the adjacency matrix, the number of paths from v_i to v_j of length 1 is the (i,j)th entry of $\mathbf{A}$.

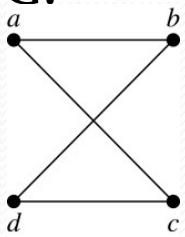
Inductive Step: For the inductive hypothesis, we assume that that the (i,j)th entry of $\mathbf{A}^r$ is the number of different paths of length r from v_i to v_j.

- Because $\mathbf{A}^{r+1} = \mathbf{A}^r \mathbf{A}$, the (i,j)th entry of $\mathbf{A}^{r+1}$ equals $b_{i1}a_{1j} + b_{i2}a_{2j} + \cdots + b_{in}a_{nj}$, where b_{ik} is the $(i,k)th$ entry of $\mathbf{A}^r$. By the inductive hypothesis, b_{ik} is the number of paths of length r from v_i to v_k.
- A path of length $r + 1$ from v_i to v_j is made up of a path of length r from v_i to some v_k, and an edge from v_k to v_j. By the product rule for counting, the number of such paths is the product of the number of paths of length r from v_i to v_k (i.e., b_{ik}) and the number of edges from from v_k to v_j (i.e, a_{kj}). The sum over all possible intermediate vertices v_k *is* $b_{i1}a_{1j} + b_{i2}a_{2j} + \cdots + b_{in}a_{nj}$.

Counting Paths between Vertices (*continued*)

Example: How many paths of length four are there from a to d in the graph G.

G



$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

adjacency matrix of G

Solution: The adjacency matrix of G (ordering the vertices as a, b, c, d) is given above. Hence the number of paths of length four from a to d is the (1, 4)th entry of $\mathbf{A}^4$. The eight paths are as:

$$\mathbf{A}^4 = \begin{bmatrix} 8 & 0 & 0 & 8 \\ 0 & 8 & 8 & 0 \\ 0 & 8 & 8 & 0 \\ 8 & 0 & 0 & 8 \end{bmatrix}$$

a, b, a, b, d	a, b, a, c, d
a, b, d, b, d	a, b, d, c, d
a, c, a, b, d	a, c, a, c, d
a, c, d, b, d	a, c, d, c, d

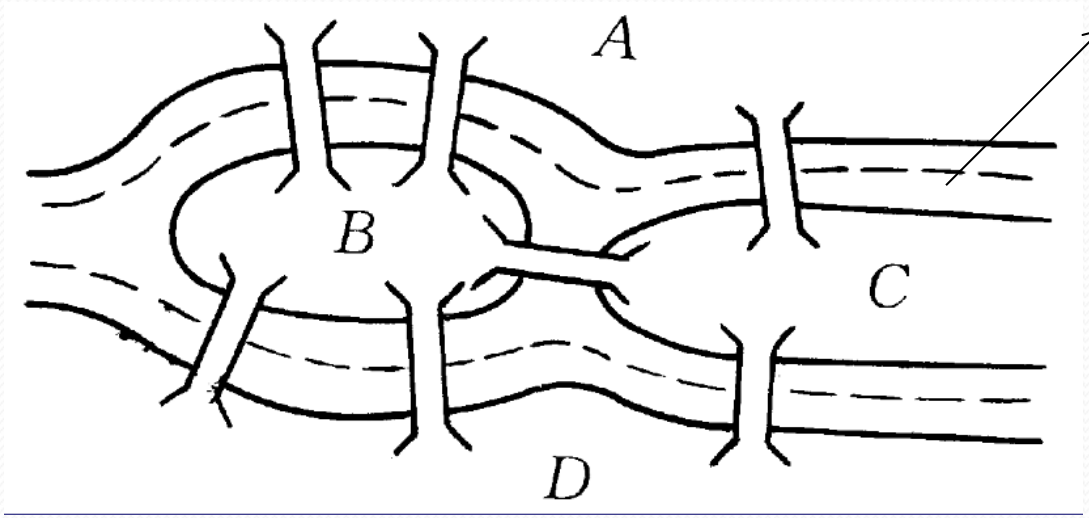
Euler and Hamiltonian Graphs
欧拉通路与哈密顿通路

Section Summary

- Euler Paths and Circuits
- Hamilton Paths and Circuits
- Applications of Hamilton Circuits

Euler Paths and Circuits

- 著名的哥尼斯堡七桥问题



The 7 Bridges of Königsberg

普雷格尔河支流

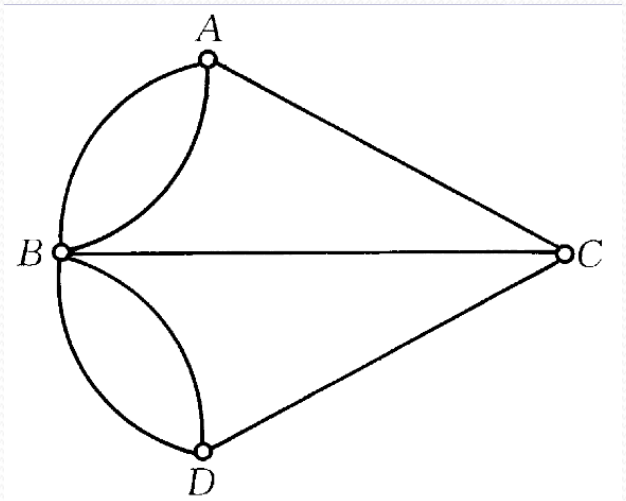


图1. 转换后的多重图

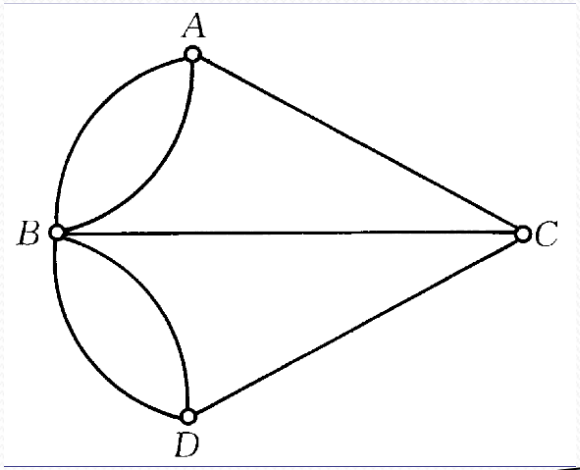
Multigraph Model of the Bridges of Königsberg

Euler Paths and Circuits



Leonard Euler
(1707-1783)

- 瑞士数学家列昂哈德•欧拉将其转换成多重图进行研究，第一次使用图论。





问题：这个多重图里是否存在着包含每一条边的简单回路?

Euler Paths and Circuits

定义

- 欧拉回路：通过图G的每条边一次且仅一次的回路。

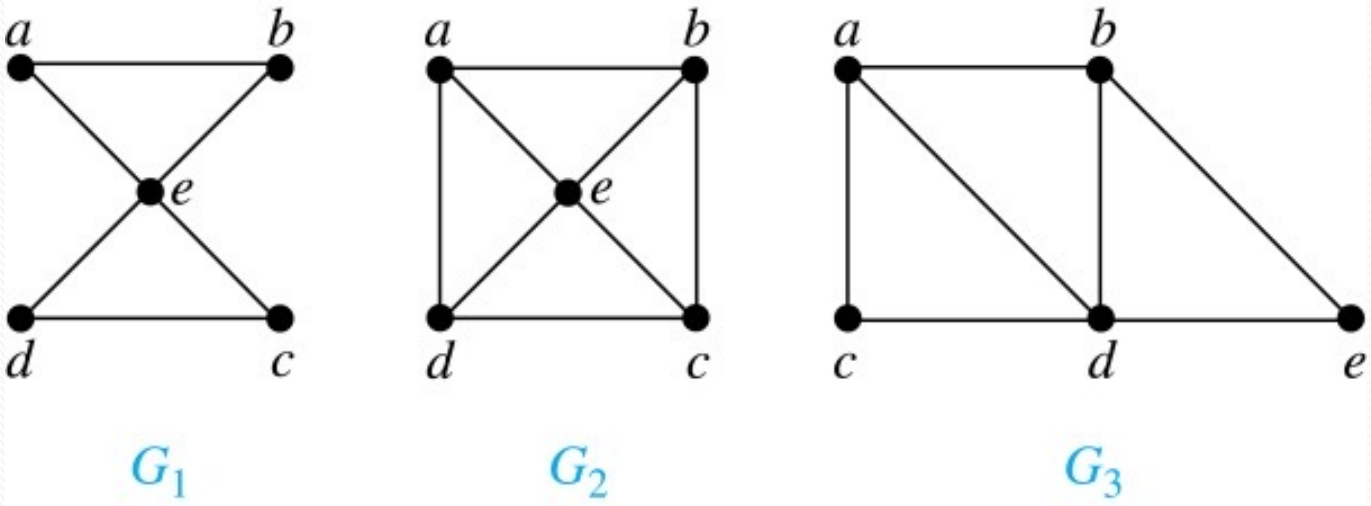
  ⟹具有欧拉回路的图称作欧拉图。

- 欧拉通路：通过图G的每条边一次且仅一次的开路。



欧拉回路与欧拉通路

例1 判断欧拉回路与欧拉通路



- 图G_1具有欧拉回路，如*a,e,c,d,e,b,a*。
- 图G_2无欧拉回路及欧拉通路。
- 图G_3具有欧拉通路，如*a,c,d,e,b,d,a,b*。

欧拉回路与欧拉通路

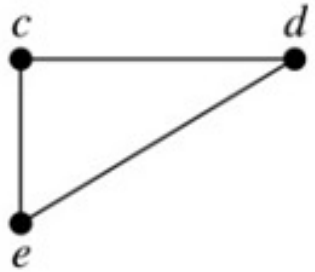
4.欧拉回路的定理及证明

- **定理1** 连通多重图具有欧拉回路当且仅当它的每个顶点都有偶数度。（充要条件）

证明：

1）必要性：

设图G是一欧拉图，α是G中的一条欧拉回路。当α通过G的任一结点时，必通过关联于该点的两条边。又因为G中的每条边仅出现一次，所以α所通过的每个结点的度数必定是偶数。



H

欧拉回路与欧拉通路

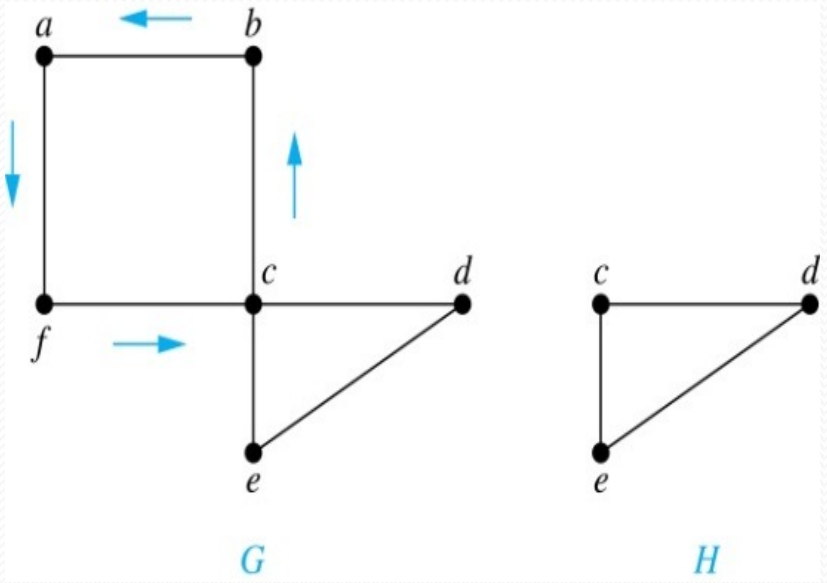
4.欧拉回路的定理及证明

充分性：

假设从结点a出发，沿着先前没有走过的新边走到另一结点（f），如此继续。由于图G的边数是有穷的，这样的通路必然会停止。

1)假如这条通路用完了所有的边，欧拉通路已经构造好了。

2）否则，考虑从G中删除已经用过的边和不关联任何剩余的边的顶点这样所得到的子图（H）。



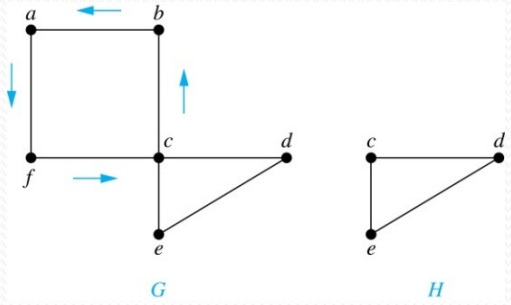
Necessary Conditions for Euler Circuits and Paths

- An Euler circuit begins with a vertex a and continues with an edge incident with a, say $\{a, b\}$. The edge $\{a, b\}$ contributes one to $\deg(a)$.
- Each time the circuit passes through a vertex it contributes two to the vertex's degree.
- Finally, the circuit terminates where it started, contributing one to $\deg(a)$. Therefore $\deg(a)$ must be even.
- We conclude that the degree of every other vertex must also be even.
- By the same reasoning, we see that the initial vertex and the final vertex of an Euler path have odd degree, while every other vertex has even degree. So, a graph with an Euler path has exactly two vertices of odd degree.
- In the next slide we will show that these necessary conditions are also sufficient conditions.

Sufficient Conditions for Euler Circuits and Paths

Suppose that G is a connected multigraph with ≥ 2 vertices, all of even degree. Let $x_0 = a$ be a vertex of even degree. Choose an edge $\{x_0, x_1\}$ incident with a and proceed to build a simple path $\{x_0, x_1\}, \{x_1, x_2\}, \ldots, \{x_{n-1}, x_n\}$ by adding edges one by one until another edge can not be added.

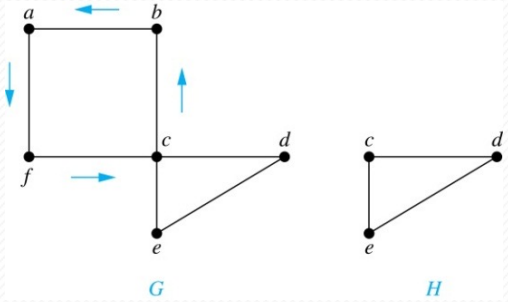
We illustrate this idea in the graph G here. We begin at a and choose the edges $\{a, f\}, \{f, c\}, \{c, b\}$, and $\{b, a\}$ in succession.



- The path begins at a with an edge of the form $\{a, x\}$; we show that it must terminate at a with an edge of the form $\{y, a\}$. Since each vertex has an even degree, there must be an even number of edges incident with this vertex. Hence, every time we enter a vertex other than a, we can leave it. Therefore, the path can only end at a.
- If all of the edges have been used, an Euler circuit has been constructed. Otherwise, consider the subgraph H obtained from G by deleting the edges already used.

In the example H consists of the vertices c, d, e.

Sufficient Conditions for Euler Circuits and Paths (*continued*)



- Because G is connected, H must have at least one vertex in common with the circuit that has been deleted.

  In the example, the vertex is *c*.

- Every vertex in H must have even degree because all the vertices in *G* have even degree and for each vertex, pairs of edges incident with this vertex have been deleted. Beginning with the shared vertex construct a path ending in the same vertex (as was done before). Then splice this new circuit into the original circuit.

  In the example, we end up with the circuit *a, f, c, d, e, c, b, a*.

- Continue this process until all edges have been used. This produces an Euler circuit. Since every edge is included and no edge is included more than once.
- Similar reasoning can be used to show that a graph with exactly two vertices of odd degree must have an Euler path connecting these two vertices of odd degree

Algorithm for Constructing an Euler Circuits

In our proof we developed this algorithms for constructing a Euler circuit in a graph with no vertices of odd degree.

procedure *Euler*(*G*: connected multigraph with all vertices of even degree)
circuit := a circuit in *G* beginning at an arbitrarily chosen vertex with edges successively added to form a path that returns to this vertex.
H := *G* with the edges of this circuit removed
while *H* has edges
    subciruit := a circuit in *H* beginning at a vertex in *H* that also is an endpoint of an edge in circuit.
    H := *H* with edges of *subciruit* and all isolated vertices removed
    circuit := *circuit* with *subcircuit* inserted at the appropriate vertex.
return *circuit*{*circuit* is an Euler circuit}

欧拉回路与欧拉通路

5.欧拉通路的定理及证明

- **定理2 连通多重图具有欧拉通路但无欧拉回路当且仅当它恰有2个奇数度顶点。**

证明：设 G 是连通多重图。

必要性

假设 G具有欧拉通路但无欧拉回路。

设这条欧拉通路从顶点 u出发，到顶点 v 结束。因为它不是欧拉回路，所以u≠v. 在欧拉通路中，除起点 u和终点 v 外，每当通路进入某个顶点，就必须从该顶点离开，因此这些中间顶点关联的边是成对出现的，所以它们的度数都是偶数。

而起点 u 多一次“离开”而少一次“进入”，终点 v 多一次“进入”而少一次“离开”，所以 u和 v 的度数都是奇数。

因此，图中恰有 2 个奇数度顶点。

欧拉回路与欧拉通路

5.欧拉通路的定理及证明

- **定理2 连通多重图具有欧拉通路但无欧拉回路当且仅当它恰有2个奇数度顶点。**

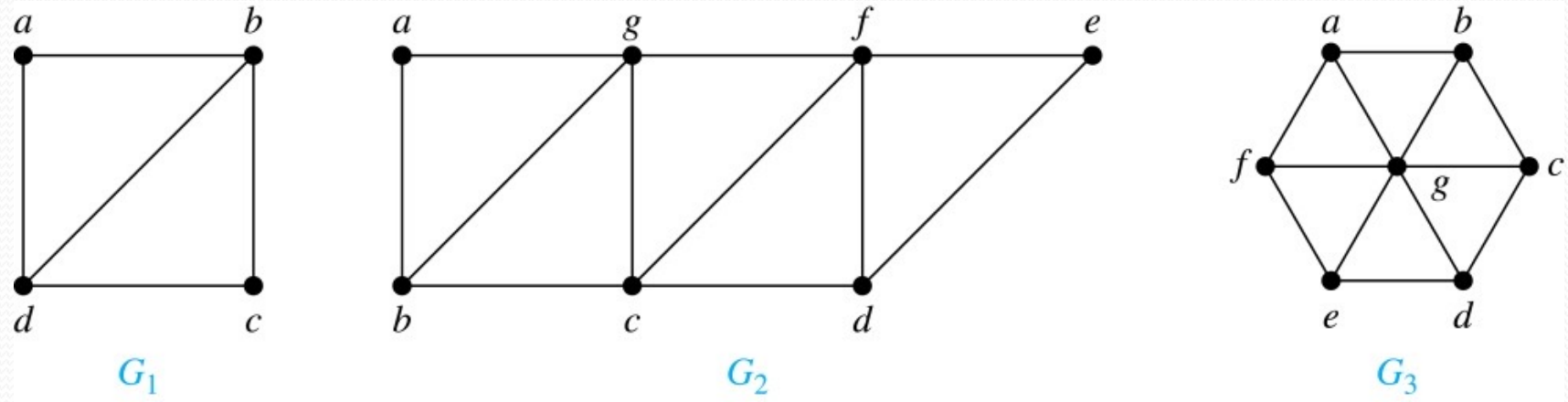
证明：假设 G 恰有 2 个奇数度顶点，记为 u和 v。

在 u 与 v 之间添加一条新边 uv，得到新图 G''。

- 此时，u 和 v 的度数各增加 1，因此它们由奇数度变成偶数度；其他顶点度数不变，仍为偶数。所以 G'中所有顶点的度数都是偶数。
- 又因为 G连通，添加一条边后 G' 仍连通。因此，根据欧拉回路定理，G'具有欧拉回路。
- 这个欧拉回路必然经过新添加的边 uv。删去这条新边后，剩余的边按照原来的顺序组成 G 中一条从 u 到 v 的欧拉通路。
- 由于 G 中存在奇数度顶点，所以它不可能有欧拉回路。
- 因此，G 具有欧拉通路但无欧拉回路。

欧拉回路与欧拉通路

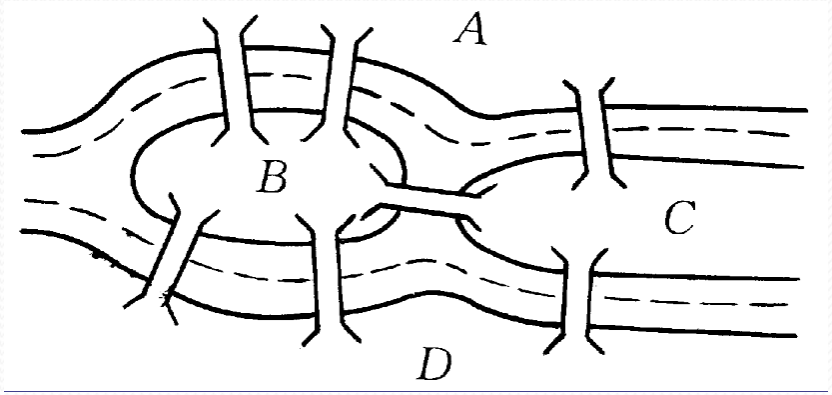
6. 判断欧拉通路

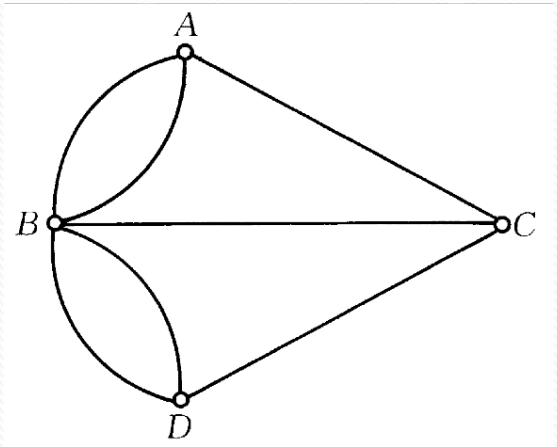


- G_1 恰有2个奇数度顶点 (b 和 d)，因此有一条欧拉通路：d, a, b, c, d, b.
- G_2 恰有2个奇数度顶点 (b 和 d)，因此有一条欧拉通路：$b, a, g, f, e, d, c, g, b, c, f, d$.
- G_3 有6个奇数度顶点，因此没有一条欧拉通路。

欧拉回路与欧拉通路

7.哥尼斯堡七桥的问题解答







1. 从镇里某点开始，恰好经过每座桥一次并返回该镇的起始点是无法实现的。⬅ 欧拉回路
2. 从镇里某点开始，不重复地旅行经过所有的桥，在镇里其他某点结束也是无法实现的。⬅ 欧拉通路

欧拉回路与欧拉通路

实际应用

- 一笔画游戏
- 邮递员送递信件
- 电路布线
- 网络组播
- 分子生物学的DNA测序

Applications of Euler Paths and Circuits

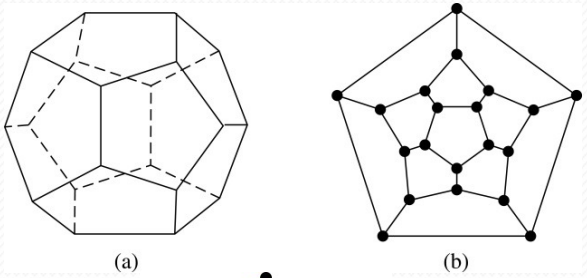
- Euler paths and circuits can be used to solve many practical problems such as finding a path or circuit that traverses each
  - street in a neighborhood,
  - road in a transportation network,
  - connection in a utility grid,
  - link in a communications network.
- Other applications are found in the
  - layout of circuits,
  - network multicasting,
  - molecular biology, where Euler paths are used in the sequencing of DNA.

Hamilton Paths and Circuits
哈密顿通路与哈密顿回路

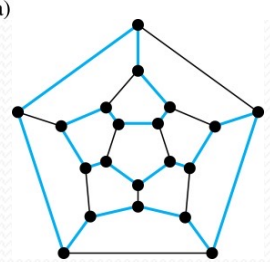
- Euler paths and circuits contained every edge only once. Now we look at paths and circuits that contain every vertex exactly once.

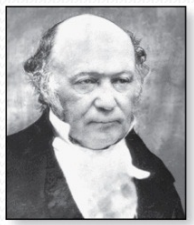
欧拉通路和欧拉回路包含多重图每一条边恰好一次的通路和回路。那包含图的每一个顶点恰好一次的简单通路和回路呢？

- William Hamilton invented the *Icosian puzzle* in 1857. It consisted of a wooden dodecahedron (with 12 regular pentagons as faces), illustrated in (a), with a peg at each vertex, labeled with the names of different cities. String was used to used to plot a circuit visiting 20 cities exactly once
- The graph form of the puzzle is given in (b).



- The solution (a Hamilton circuit) is given here.





William Rowan Hamilton (1805- 1865)

爱尔兰数学家 威廉 罗万 哈密顿爵士

Hamilton Paths and Circuits

Definition: A simple path in a graph G that passes through every vertex exactly once is called a *Hamilton path*, and a simple circuit in a graph G that passes through every vertex exactly once is called a *Hamilton circuit*.

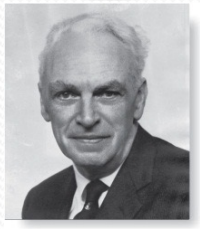
在图$G = (V, E)$ 里，若$V = \{x_0, x_1, \ldots, x_{n-1}, x_n\}$ 并且对$0 \leq i < j \leq n$来说有$x_i \neq x_j$，则通路$x_0, x_1, \ldots, x_{n-1}, x_n$ 称为**哈密顿通路**。

在图$G = (V, E)$ 里，若$x_0, x_1, \ldots, x_{n-1}, x_n$ 是哈密顿通路，则$x_0, x_1, \ldots, x_{n-1}, x_n, x_0$（其中$n > 1$）称为**哈密顿回路**。

That is, a simple path $x_0, x_1, \ldots, x_{n-1}, x_n$ in the graph $G = (V, E)$ is called a Hamilton path if $V = \{x_0, x_1, \ldots, x_{n-1}, x_n\}$ and $x_i \neq x_j$ for $0 \leq i < j \leq n$, and the simple circuit $x_0, x_1, \ldots, x_{n-1}, x_n, x_0$ (with $n > 0$) is a Hamilton circuit if $x_0, x_1, \ldots, x_{n-1}, x_n$ is a Hamilton path.



Gabriel Andrew Dirac
(1925-1984)



Øysten Ore
(1899-1968)

- Unlike for an Euler circuit, no simple necessary and sufficient conditions are known for the existence of a Hamiton circuit.
- However, there are some useful necessary conditions. We describe two of these now.

  Dirac's Theorem: If G is a simple graph with $n \geq 3$ vertices such that the degree of every vertex in G is $\geq n/2$, then G has a Hamilton circuit.

  Ore's Theorem: If G is a simple graph with $n \geq 3$ vertices such that $\deg(u) + \deg(v) \geq n$ for every pair of nonadjacent vertices, then G has a Hamilton circuit.

  狄拉克定理：如果G是带n个顶点的连通简单图，其中$n \geq 3$，并且G中每个顶点的度都至少为 n/2，则G有哈密顿回路。

  奥尔定理：如果G是带n个顶点的连通简单图，其中$n \geq 3$，并且对于G中每一对不相邻的顶点u和v来说，都有$\deg(u) + \deg(v) \geq n$，则G有哈密顿回路。

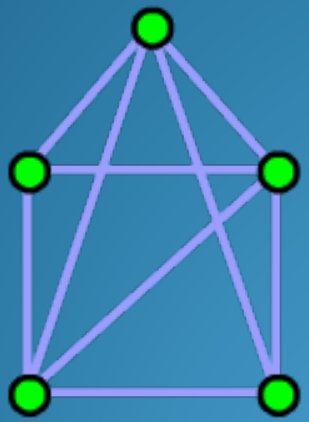
Applications of Hamilton Paths and Circuits

- Applications that ask for a path or a circuit that visits each intersection of a city, each place pipelines intersect in a utility grid, or each node in a communications network exactly once, can be solved by finding a Hamilton path in the appropriate graph.
- The famous *traveling salesperson problem* (*TSP*) asks for the shortest route a traveling salesperson should take to visit a set of cities. This problem reduces to finding a Hamilton circuit such that the total sum of the weights of its edges is as small as possible.
- A family of binary codes, known as *Gray codes*, which minimize the effect of transmission errors, correspond to Hamilton circuits in the n-cube Q_n. (*See the text for details*.)

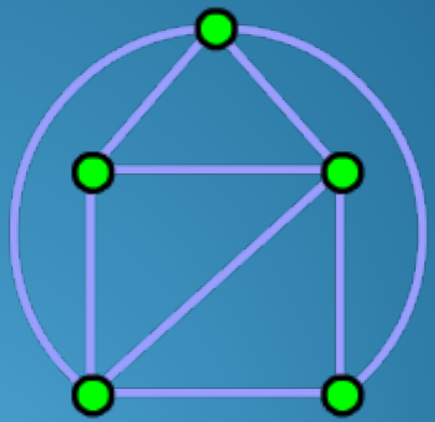
平面图

一、平面图的定义

定义 一个图G若能画在平面上，它的边除在端点处外互不交叉，则称G为平面图。画出的没有边交叉的图解称为G的一个平面嵌入。



(a)



(b)

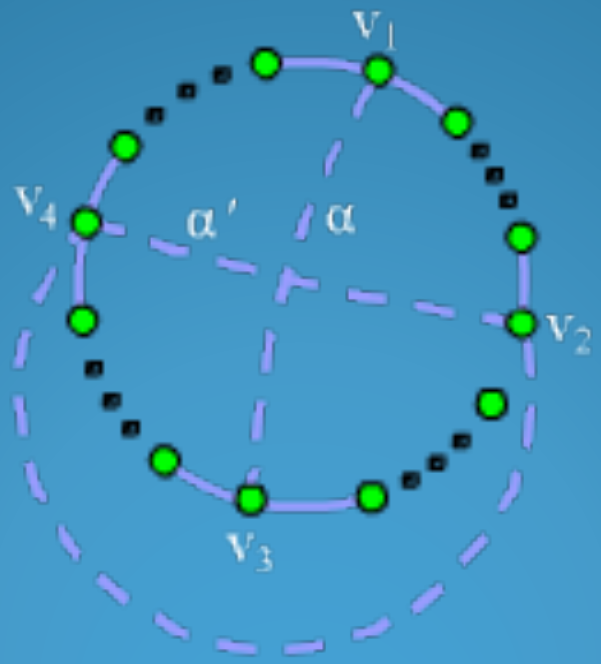
例1 图（a）是平面图，（b）是该图的一个平面嵌入。

二、平面图的判别

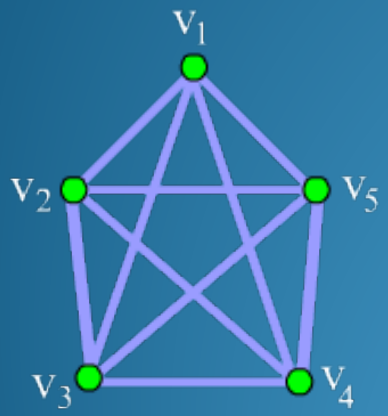
1．简单、直观判别法

设G是画于平面上的一个图，$\sigma=v_1\cdots v_2\cdots v_3\cdots v_4\cdots v_1$是G中任意一个环。$\alpha=v_1\cdots v_3$和$\alpha'=v_2\cdots v_4$是G中任意两条边无公共结点的真路。

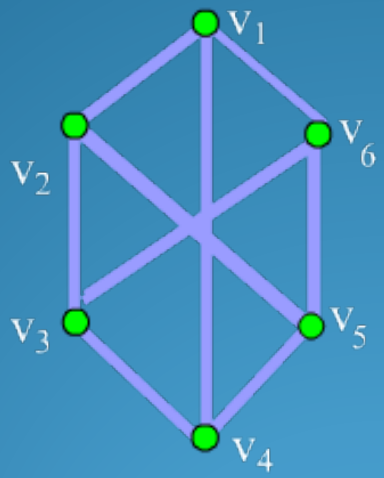
我们往往可以用观察的方法来判别一个图是否为平面图。



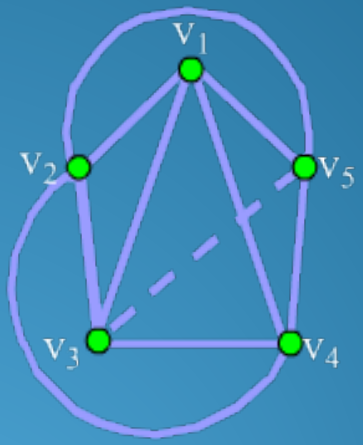
用上述简单、直观的方法判别下图中的两图是否平面图。



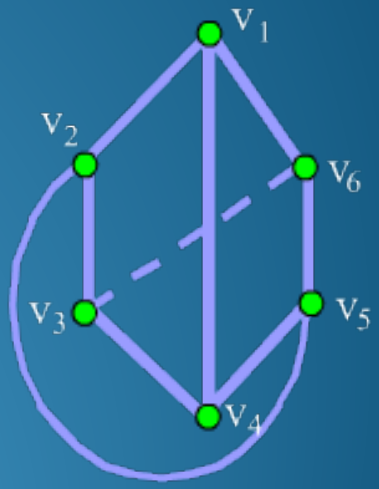
K_5



$K_{3,3}$



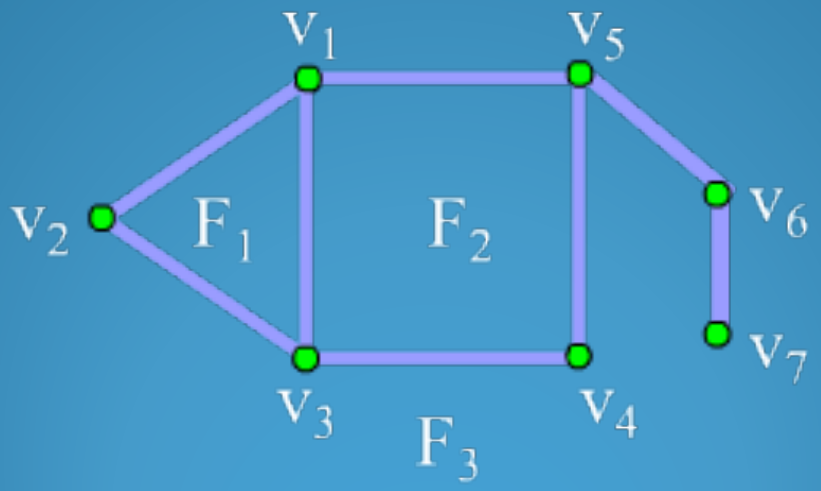
(a)



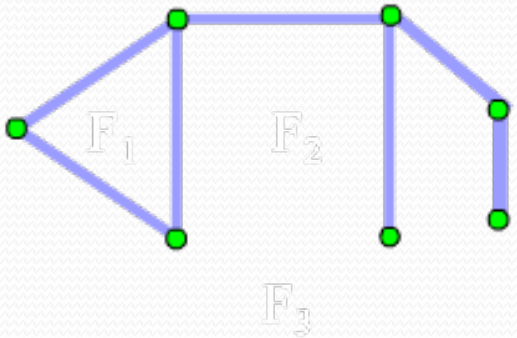
(b)

2. 欧拉公式判断法

定义 设G是一个连通平面图，G的边将G所在的平面划分成若干个区域，每一个区域称为G的一个面。其中面积无限的区域称为无限面。面积有限的区域称为有限面。包围每个面的所有边构成的回路称为面的边界。



定理 设G是一连通平面图，有n个结点，m条边，K个面，则 $n-m+K=2$



此定理中的公式称为欧拉公式。

证明 （对边数m进行归纳）

当m=1时，定理显然成立。

假设定理对m－1条边的任何连通平面图均成立，设G是一具有n个结点，m条边，K个面的连通平面图（m≥2）

若G中没有环，则G是一棵树，K=1，且m=n–1，于是

n–m+K=n–（n–1）+1=2。

若G中有环，则去掉G的任一环上的一条边e，剩下的图G′仍连通，有n个结点，K－1个面，m－1条边，

由归纳假设G′中欧拉公式成立。

因此n－（m－1）+（K－1）=2，即n－m+K=2。

定理 设G是一（n，m）的连通平面图，m≥2，

则 m≤3n－6 （*）

证明 当m=2时，因G是简单图必无环，

所以n=3， 上式成立。

当m>2时，每个面至少由三条边围成，因此各面边界的边数之和Σ≥3K。

另一方面，因为有些边同时在两个面的边界中，在Σ中作了重复计数，所以Σ≤2m。

于是得到不等式 **3K≤2m**， 即 **K≤2m/3**。

根据欧拉公式，$n-m+\frac{2}{3}m\geq 2$

因此 **m≤3n–6**

推论 设G是一个（n，m）的连通平面图。

（1）若每个面至少由三条边围成，则$m \leq 3n-6$；

（2）若每个面至少由四条边围成，则$m \leq 2n-4$。

书上的推论：若连通可平面简单图有m条边和n个顶点，$n \geq 3$并且没有长度为3的回路，则$m \leq 2n-4$。

利用定理8-15判别图下图中的K_5和$K_{3,3}$是否非平面图。

解 ① 图K_5中n=5，m=10，3n－6=9。

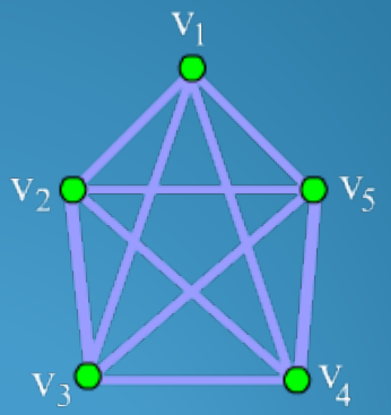
显然m $\not\leqslant$3n－6。因此k_5不是平面图。

② 图$K_{3,3}$中n=6，m=9，3n－6=12。

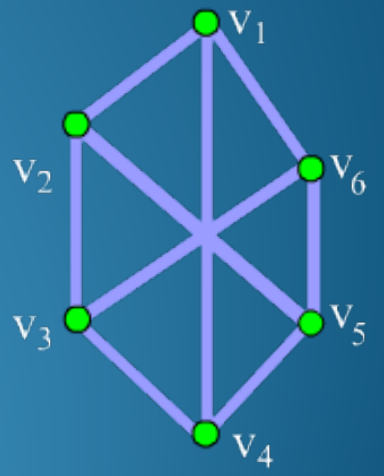
因此 m≤3n－6。满足（*）式，故无法判定$K_{3,3}$是非平面图。

实际上，利用$K_{3,3}$是二部图，可以证明，如果它是平面图，则应满足m≤2n－4，但它并不满足。

因此$K_{3,3}$不是平面图。



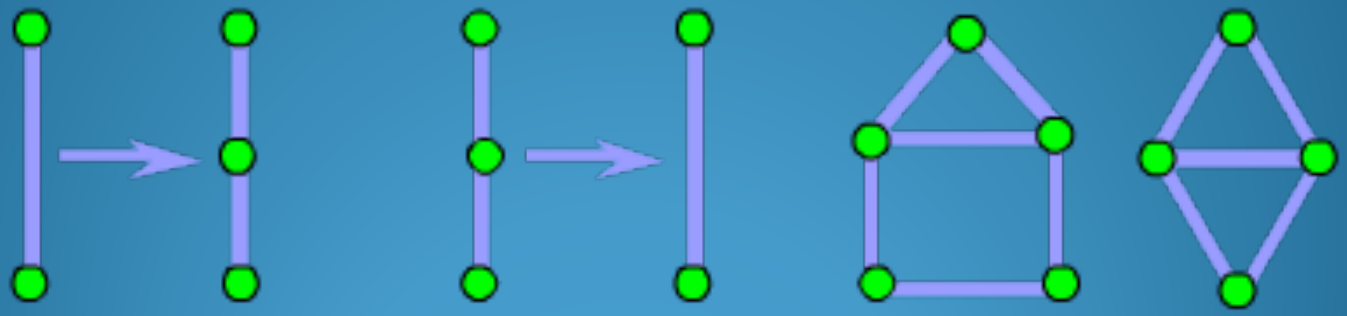
K_5



$K_{3,3}$

3．库拉托斯基定理判别法

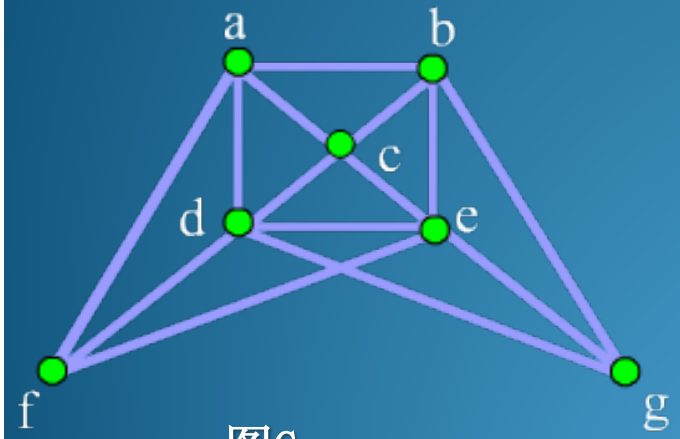
定义　　如果两个图G_1和G_2是同构的，或者通过反复插入或删除度为2的结点，它们能变成同构的图，则称G_1和G_2在度为2的结点内同构（同胚的）。



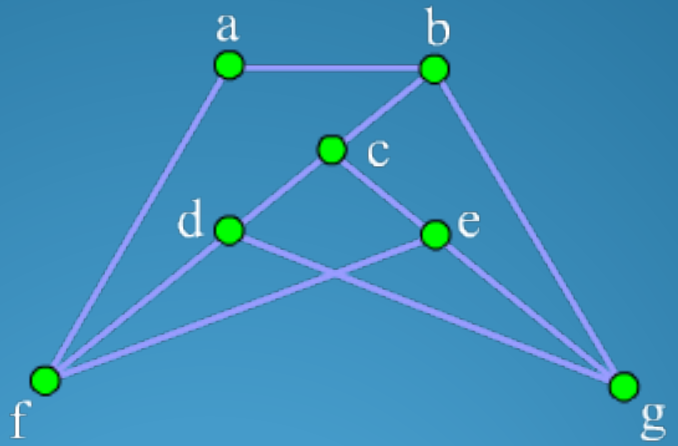
定理 （库拉托斯基定理）

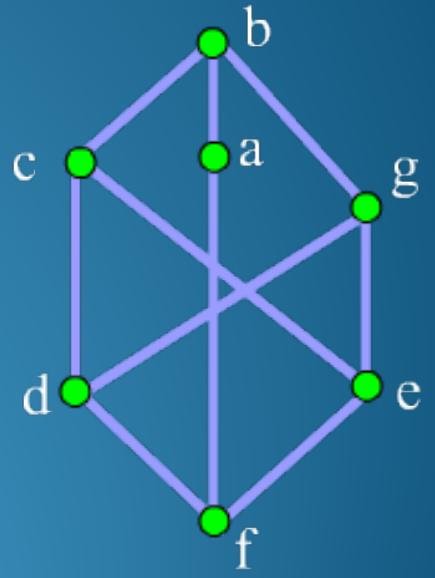
一个图是平面图的充要条件是它不包含在度为2的结点内与K_5同构的子图，也不包含在度为2的结点内与$K_{3,3}$同构的子图。

判别图G是否非平面图。



图G

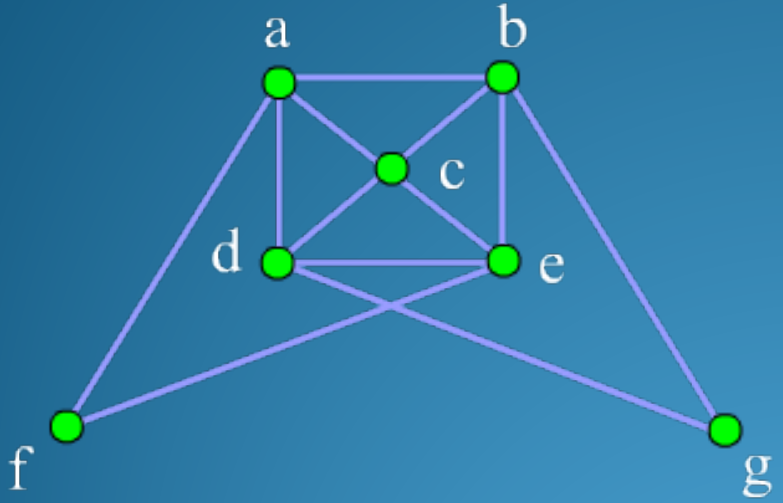


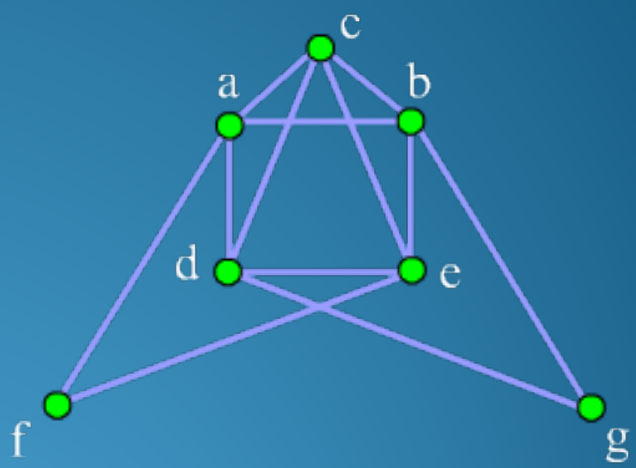


解法一 （1）去掉图G中边

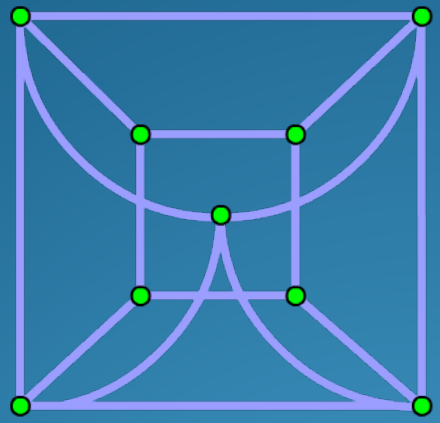
{a，c}，{a，d}，{d，e}，{b，e}，

解法二　（1）去掉图中边{d，f}和{e，g}，

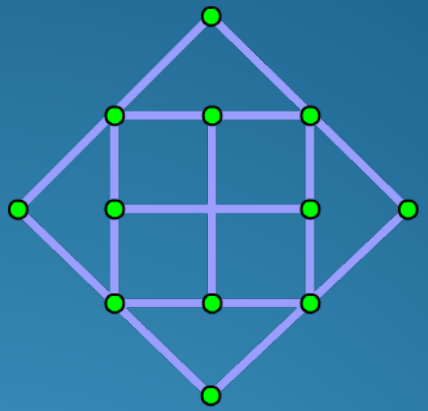




1．用简单、直观判别法判断下图所给出的两个图是否平面图。

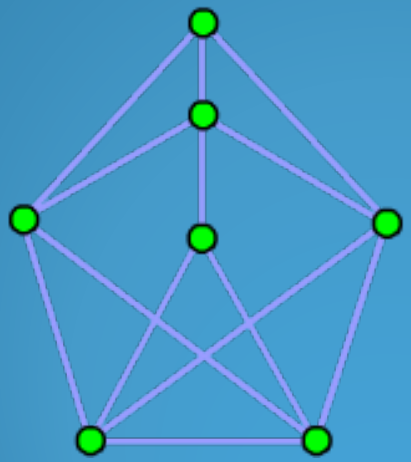


(Y)



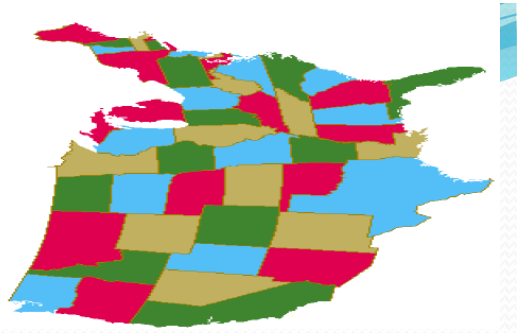
(Y)

2．用库拉托斯基定理判别下图是否平面图。



(N)

平面图与地图着色问题



与平面图有关的是著名的“4色猜想”，问题起源于1852年英国一个青年在画地图时发现：如果使相邻两国着上不同的颜色，那么，画任何一张地图只需要4种颜色就够了。

- 许多的实际问题都可以转化为图的着色问题。
- 图的着色有两种类型，一种是给某图的顶点着色，另一种是给某图的边着色。
- 若要求图的邻接顶点着不同色，问至少要几种颜色？这就是所谓的点着色问题。

地图着色问题实际上就是平面图的点着色问题。

四色定理：平面图着色数不超过4。

四色定理至今没有简单的理论证明，1976年美国伊利诺斯大学的两位数学家肯尼斯·阿佩尔和沃尔夫刚·黑肯用计算机运行了将近1200小时证明了四色猜想成立。

希伍德五色定理：任一连通简单平面图着色数不超过5。

1890年希伍德证明了任何连通简单平面图都是可5着色的。

定义 设G是一个平面图，对G的每一个结点指定一种颜色，使相邻接结点具有不同颜色，称为对图G的正常着色，简称着色。

如果图G着色时用了n种颜色，则称图G为n色的。对图G着色时，需要的最少颜色数，称为G的着色数，记为$\chi(G)$。

图着色的应用

例 如何安排我们大学的期末考试，使得没有学生在同一时间有两门考试？

解：这样的安排问题可以用图模型来解决。

用顶点表示课程，若在两个顶点所表示的课程里有公共的学生，则在这两个顶点之间有边。用不同的颜色来表示期末考试的每个时间段。考试的安排就对应于所关联的图的着色。